

ROLE: You are a senior Next.js engineer and accessibility reviewer.

CONTEXT: We are building a Campus Companion web app for first-year students

CONSTRAINTS: - Next.js (App Router), TypeScript - Serverless backend: Supabase - Accessibility: WCAG AA principles, keyboard-first - Data: fictional only (no real personal data)

TASK: [

a sign up and login page which appears left to the gear icon, when a user signs in successfully, it should turn into My Account, My Account should appear left of the gear icon with the following options [My account which moves the user to a larger page about their account, forgot password, signout] (This signup should ask the user for their email, name and password, when the user wants to login in they can login with email and their password the password must match and should be encrypted in supabase, users should then have saved data for things such as if they are already interested in an event or for their timetable)

Notifications a notification bell icon next to the my account or login/signup where it only appears once a user has logged in, and when they receive a notification it gets a red dot

Timetable should follow an academic weekly table where you are able to view next week or previous week table, showing your modules, lecturers name, room location, a group identifier for the timetable make up 3, and a week view layout from 8:00 to 17:00 (This timetable should be customisable in supabase, and users should have the option to set one of the group identifier as their timetable),

Events should have the event name, an image, the date and time, a location, and a tag, you should be able to click onto the event and read a longer description outlining the event, and you should be able to press a button to show

whether you're interested(You should be able to add more events directly from supabase, for students who are interested in going you should be able to view their name and email in supabase as well as which event they are interested in),

an open map which i can later customise for our college campus with a tag so its easier to see the pointer needs to be a visually contrasting colour,

Society events students should be able to join societies and receive an update about upcoming events in there society and you should be able to press a button to show whether you're interested, students should still be able to view other society events but only receive notifications on there society (You should be able to add more society events directly from supabase, for students who are interested in going you should be able to view their name and email in supabase as well as which society event they are interested in, supabase also needs to update which society they are currently in can be more than one) ,

lost and found should just be a lost and found where students can list either found items or lost items (shows up in supabase only),

helpdesk students can send help messages (shows up in supabase only),

and
accessibility settings such as more contrast, text size toggles]

Timetable, Events, Society events and Map should appear as separate pages

Lost and Found, Helpdesk should both appear under the same page with a sidebar to choose which one
accessibility settings should appear as a gear icon top right where you can choose your settings

OUTPUT FORMAT: Provide (1) files to create/edit, (2) code only, (3) brief test steps.

SELF-CHECK: After code, list likely accessibility issues and fixes.

tailwind css, the design should be modern easy to understand and access and clear and teach me the set up from start i am using vscode

I'm putting this together as a full starter build so you can drop it into VS Code and wire it up step by step. I'm checking the current Next.js App Router and Supabase docs first so the structure and auth flow match the current setup.

I've got the current docs I need. I'm shaping this as a realistic starter app with Supabase auth, RLS, saved user preferences, protected pages, and a keyboard-friendly UI so you can build it cleanly in VS Code.

Thought for 9m 23s >

< 1/2 >      ...

keep going

I've got the auth pattern pinned down from the current Supabase SSR docs, so I'm wiring this the current way instead of the older auth-helpers approach. Now I'm packaging it as a VS Code-friendly starter with file-by-file code, Supabase SQL, and test steps.

Thought for 9m 51s >

This starter follows the current App Router `create-next-app` flow and Supabase's current SSR setup using `@supabase/ssr`, cookie-based clients, and `proxy.ts`. Supabase Auth stores password hashes with `bcrypt`, so you do not manually encrypt passwords in your Next.js code. Next.js +3

1) Files to create/edit

```
campus-companion/  
├ .env.local  
├ proxy.ts  
├ supabase/  
│   └ schema.sql  
├ src/  
│   └ app/
```



</> Bash

`.env.local`

</> env

`supabase/schema.sql`

```
create extension if not exists pgcrypto;

drop trigger if exists on_auth_user_created on auth.users;
drop function if exists public.handle_new_user cascade;
drop function if exists public.handle_new_society_event_notifications cascade;

drop table if exists public.helpdesk_messages cascade;
drop table if exists public.lost_found_items cascade;
drop table if exists public.notifications cascade;
drop table if exists public.society_event_interests cascade;
drop table if exists public.society_events cascade;
drop table if exists public.society_memberships cascade;
drop table if exists public.societies cascade;
drop table if exists public.event_interests cascade;
drop table if exists public.events cascade;
drop table if exists public.accessibility_preferences cascade;
drop table if exists public.map_locations cascade;
drop table if exists public.timetable_entries cascade;
drop table if exists public.profiles cascade;
drop table if exists public.timetable_groups cascade;

drop type if exists public.text_scale_option cascade;
drop type if exists public.lost_found_type cascade;

create type public.text_scale_option as enum ('sm', 'base', 'lg');
create type public.lost_found_type as enum ('lost', 'found');

create table public.timetable_groups (
  id uuid primary key default gen_random_uuid(),
  code text not null unique,
  title text not null,
  created_at timestamptz not null default now()
);

create table public.profiles (
```

```
id uuid primary key references auth.users(id) on delete cascade,
full_name text not null,
email text not null unique,
selected_timetable_group_id uuid references public.timetable_groups(id) on delete cascade,
created_at timestamptz not null default now()
);

create table public.accessibility_preferences (
  user_id uuid primary key references auth.users(id) on delete cascade,
  high_contrast boolean not null default false,
  text_scale public.text_scale_option not null default 'base',
  updated_at timestamptz not null default now()
);

create table public.timetable_entries (
  id uuid primary key default gen_random_uuid(),
  group_id uuid not null references public.timetable_groups(id) on delete cascade,
  day_of_week integer not null check (day_of_week between 1 and 5),
  start_time time not null,
  end_time time not null,
  module_code text not null,
  module_name text not null,
  lecturer_name text not null,
  room_name text not null,
  created_at timestamptz not null default now()
);

create table public.events (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  image_url text not null,
  start_at timestamptz not null,
  end_at timestamptz not null,
  location text not null,
  tag text not null,
  short_description text not null,
  description text not null,
  created_at timestamptz not null default now()
);

create table public.event_interests (
  id uuid primary key default gen_random_uuid(),
  event_id uuid not null references public.events(id) on delete cascade,
  user_id uuid not null references auth.users(id) on delete cascade,
  created_at timestamptz not null default now(),
  unique (event_id, user_id)
);

create table public.societies (
  id uuid primary key default gen_random_uuid(),
  name text not null unique,
  tag text not null,
```

```
description text not null,  
hero_image_url text not null,  
created_at timestampz not null default now()  
);  
  
create table public.society_memberships (  
  id uuid primary key default gen_random_uuid(),  
  society_id uuid not null references public.societies(id) on delete cascade,  
  user_id uuid not null references auth.users(id) on delete cascade,  
  created_at timestampz not null default now(),  
  unique (society_id, user_id)  
);  
  
create table public.society_events (  
  id uuid primary key default gen_random_uuid(),  
  society_id uuid not null references public.societies(id) on delete cascade,  
  name text not null,  
  image_url text not null,  
  start_at timestampz not null,  
  end_at timestampz not null,  
  location text not null,  
  tag text not null,  
  description text not null,  
  created_at timestampz not null default now()  
);  
  
create table public.society_event_interests (  
  id uuid primary key default gen_random_uuid(),  
  society_event_id uuid not null references public.society_events(id) on delete c  
  user_id uuid not null references auth.users(id) on delete cascade,  
  created_at timestampz not null default now(),  
  unique (society_event_id, user_id)  
);  
  
create table public.notifications (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references auth.users(id) on delete cascade,  
  title text not null,  
  body text not null,  
  href text,  
  is_read boolean not null default false,  
  created_at timestampz not null default now()  
);  
  
create table public.map_locations (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  tag text not null,  
  latitude numeric(9, 6) not null,  
  longitude numeric(9, 6) not null,  
  description text,  
  created_at timestampz not null default now()
```

```
);

create table public.lost_found_items (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  item_type public.lost_found_type not null,
  title text not null,
  description text not null,
  location text not null,
  contact_email text not null,
  created_at timestamptz not null default now()
);

create table public.helpdesk_messages (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  subject text not null,
  category text not null,
  message text not null,
  created_at timestamptz not null default now()
);

create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
  insert into public.profiles (id, full_name, email)
  values (
    new.id,
    coalesce(new.raw_user_meta_data ->> 'full_name', 'New Student'),
    new.email
  );

  insert into public.accessibility_preferences (user_id)
  values (new.id);

  return new;
end;
$$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute procedure public.handle_new_user();

create or replace function public.handle_new_society_event_notifications()
returns trigger
language plpgsql
security definer
set search_path = public
```



```
as $$
begin
    insert into public.notifications (user_id, title, body, href)
    select
        sm.user_id,
        'New society event',
        new.name || ' was added for a society you joined.',
        '/societies'
    from public.society_memberships sm
    where sm.society_id = new.society_id;

    return new;
end;
$$;

create trigger on_society_event_created
after insert on public.society_events
for each row execute procedure public.handle_new_society_event_notifications();

alter table public.profiles enable row level security;
alter table public.accessibility_preferences enable row level security;
alter table public.timetable_groups enable row level security;
alter table public.timetable_entries enable row level security;
alter table public.events enable row level security;
alter table public.event_interests enable row level security;
alter table public.societies enable row level security;
alter table public.society_memberships enable row level security;
alter table public.society_events enable row level security;
alter table public.society_event_interests enable row level security;
alter table public.notifications enable row level security;
alter table public.map_locations enable row level security;
alter table public.lost_found_items enable row level security;
alter table public.helpdesk_messages enable row level security;

create policy "profiles_select_own"
on public.profiles
for select
to authenticated
using (auth.uid() is not null and auth.uid() = id);

create policy "profiles_update_own"
on public.profiles
for update
to authenticated
using (auth.uid() is not null and auth.uid() = id)
with check (auth.uid() is not null and auth.uid() = id);

create policy "accessibility_select_own"
on public.accessibility_preferences
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);
```

```
create policy "accessibility_insert_own"
on public.accessibility_preferences
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "accessibility_update_own"
on public.accessibility_preferences
for update
to authenticated
using (auth.uid() is not null and auth.uid() = user_id)
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "timetable_groups_public_read"
on public.timetable_groups
for select
to anon, authenticated
using (true);

create policy "timetable_entries_public_read"
on public.timetable_entries
for select
to anon, authenticated
using (true);

create policy "events_public_read"
on public.events
for select
to anon, authenticated
using (true);

create policy "event_interests_select_own"
on public.event_interests
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "event_interests_insert_own"
on public.event_interests
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "event_interests_delete_own"
on public.event_interests
for delete
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "societies_public_read"
on public.societies
```

```
for select
to anon, authenticated
using (true);

create policy "society_events_public_read"
on public.society_events
for select
to anon, authenticated
using (true);

create policy "society_memberships_select_own"
on public.society_memberships
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "society_memberships_insert_own"
on public.society_memberships
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "society_memberships_delete_own"
on public.society_memberships
for delete
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "society_event_interests_select_own"
on public.society_event_interests
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "society_event_interests_insert_own"
on public.society_event_interests
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "society_event_interests_delete_own"
on public.society_event_interests
for delete
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "notifications_select_own"
on public.notifications
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);
```

```
create policy "notifications_update_own"
on public.notifications
for update
to authenticated
using (auth.uid() is not null and auth.uid() = user_id)
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "map_locations_public_read"
on public.map_locations
for select
to anon, authenticated
using (true);

create policy "lost_found_insert_own"
on public.lost_found_items
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "lost_found_select_own"
on public.lost_found_items
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "helpdesk_insert_own"
on public.helpdesk_messages
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "helpdesk_select_own"
on public.helpdesk_messages
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create view public.event_interest_admin_view as
select
    e.name as event_name,
    p.full_name,
    p.email,
    ei.created_at as interested_at
from public.event_interests ei
join public.events e on e.id = ei.event_id
join public.profiles p on p.id = ei.user_id;

create view public.society_event_interest_admin_view as
select
    se.name as society_event_name,
    s.name as society_name,
    p.full_name,
```

```

    p.email,
    sei.created_at as interested_at
from public.society_event_interests sei
join public.society_events se on se.id = sei.society_event_id
join public.societies s on s.id = se.society_id
join public.profiles p on p.id = sei.user_id;

create view public.society_members_admin_view as
select
    s.name as society_name,
    p.full_name,
    p.email,
    sm.created_at as joined_at
from public.society_memberships sm
join public.societies s on s.id = sm.society_id
join public.profiles p on p.id = sm.user_id;

insert into public.timetable_groups (code, title)
values
    ('G1', 'Group 1'),
    ('G2', 'Group 2'),
    ('G3', 'Group 3');

insert into public.timetable_entries (
    group_id,
    day_of_week,
    start_time,
    end_time,
    module_code,
    module_name,
    lecturer_name,
    room_name
)
values
    ((select id from public.timetable_groups where code = 'G1'), 1, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G1'), 2, '12:00', '14:00',
    ((select id from public.timetable_groups where code = 'G1'), 4, '10:00', '12:00',
    ((select id from public.timetable_groups where code = 'G1'), 5, '14:00', '16:00',

    ((select id from public.timetable_groups where code = 'G2'), 1, '10:00', '12:00',
    ((select id from public.timetable_groups where code = 'G2'), 3, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G2'), 4, '13:00', '15:00',
    ((select id from public.timetable_groups where code = 'G2'), 5, '11:00', '13:00',

    ((select id from public.timetable_groups where code = 'G3'), 2, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G3'), 3, '12:00', '14:00',
    ((select id from public.timetable_groups where code = 'G3'), 4, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G3'), 5, '13:00', '15:00',

insert into public.events (
    name,
    image_url,

```

```

    start_at,
    end_at,
    location,
    tag,
    short_description,
    description
)
values
(
    'Welcome Fair',
    'https://images.unsplash.com/photo-1529156069898-49953e39b3ac?q=80&w=1200&auto=compress&format=webp',
    now() + interval '7 days',
    now() + interval '7 days 3 hours',
    'Main Hall',
    'Campus Life',
    'Meet societies, student services, and other first years.',
    'A fictional welcome fair for first year students. Explore student services,
),
(
    'Study Skills Bootcamp',
    'https://images.unsplash.com/photo-1522202176988-66273c2fd55f?q=80&w=1200&auto=compress&format=webp',
    now() + interval '10 days',
    now() + interval '10 days 2 hours',
    'Learning Centre',
    'Academic',
    'Time management, note taking, and exam prep tips.',
    'A practical workshop covering revision structure, note making, and weekly pl
),
(
    'Career Coffee Morning',
    'https://images.unsplash.com/photo-1515169067868-5387ec356754?q=80&w=1200&auto=compress&format=webp',
    now() + interval '14 days',
    now() + interval '14 days 2 hours',
    'Student Hub Café',
    'Careers',
    'Informal networking and CV advice.',
    'Meet the fictional careers team for a casual networking event with coffee an
);

insert into public.societies (
    name,
    tag,
    description,
    hero_image_url
)
values
(
    'Finance Society',
    'Careers',
    'Networking, trading games, and guest speakers for students interested in fin
    'https://images.unsplash.com/photo-1554224155-8d04cb21cd6c?q=80&w=1200&auto=compress&format=webp',
),

```

```

(
  'Debate Society',
  'Community',
  'Weekly debates, public speaking practice, and competitions.',
  'https://images.unsplash.com/photo-1515165562835-c3b8c2b9d4c0?q=80&w=1200&aut
),
(
  'Tech Makers',
  'Innovation',
  'Friendly coding sessions and beginner workshops for practical projects.',
  'https://images.unsplash.com/photo-1516321318423-f06f85e504b3?q=80&w=1200&aut
);

insert into public.society_events (
  society_id,
  name,
  image_url,
  start_at,
  end_at,
  location,
  tag,
  description
)
values
(
  (select id from public.societies where name = 'Finance Society'),
  'ETF Basics Night',
  'https://images.unsplash.com/photo-1559526324-4b87b5e36e44?q=80&w=1200&auto=f
now() + interval '5 days',
now() + interval '5 days 2 hours',
  'Room F12',
  'Workshop',
  'An introductory session on ETFs, long term investing, and financial literacy
),
(
  (select id from public.societies where name = 'Debate Society'),
  'Mini Debate Tournament',
  'https://images.unsplash.com/photo-1517048676732-d65bc937f952?q=80&w=1200&aut
now() + interval '8 days',
now() + interval '8 days 3 hours',
  'Lecture Theatre 2',
  'Competition',
  'Students form quick teams and take part in friendly debate rounds with start
),
(
  (select id from public.societies where name = 'Tech Makers'),
  'Campus App Jam',
  'https://images.unsplash.com/photo-1515879218367-8466d910aaa4?q=80&w=1200&aut
now() + interval '12 days',
now() + interval '12 days 3 hours',
  'Innovation Lab',
  'Build',

```

```

    'A beginner friendly build session where students prototype useful campus too
  );

insert into public.map_locations (
  name,
  tag,
  latitude,
  longitude,
  description
\
</> SQL
('Student Hub', 'Support', 53.349220, -6.259750, 'Advice desk, careers support,
('Main Hall', 'Events', 53.350020, -6.261100, 'Large space for fairs, talks, an
('Innovation Lab', 'Tech', 53.349420, -6.258900, 'Creative project and workshop

```



proxy.ts

```

import { type NextRequest } from "next/server";
import { updateSession } from "@lib/supabase/proxy";

export async function proxy(request: NextRequest) {
  return updateSession(request);
}

export const config = {
  matcher: [
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)
  ],
</> TypeScript

```



src/lib/supabase/client.ts

```

import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
</> TypeScript

```



src/lib/supabase/server.ts


```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) => {
              cookieStore.set(name, value, options);
            });
          } catch {
            // ignored in Server Components because proxy.ts refreshes the session
          }
        },
      },
    },
  );
}
```

</> TypeScript



src/lib/supabase/proxy.ts

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value, options }) => {

```

```
        request.cookies.set({ name, value, ...options });
    });

    supabaseResponse = NextResponse.next({ request });

    cookiesToSet.forEach(({ name, value, options }) => {
        supabaseResponse.cookies.set(name, value, options);
    });
    },
    },
    },
);

await supabase.auth.getClaims();
```

</> TypeScript



src/lib/types.ts

```
export type TextScale = "sm" | "base" | "lg";

export type AccessibilityPreferences = {
    high_contrast: boolean;
    text_scale: TextScale;
};

export type TimetableGroup = {
    id: string;
    code: string;
    title: string;
};

export type TimetableEntry = {
    id: string;
    group_id: string;
    day_of_week: number;
    start_time: string;
    end_time: string;
    module_code: string;
    module_name: string;
    lecturer_name: string;
    room_name: string;
};

export type MapLocation = {
    id: string;
    name: string;
    tag: string;
```

```
latitude: number;  
longitude: number;
```

⌘ TypeScript



src/app/globals.css

```
@import "leaflet/dist/leaflet.css";  
@import "tailwindcss";  
  
:root {  
  --bg: 245 247 251;  
  --surface: 255 255 255;  
  --surface-2: 241 245 249;  
  --fg: 15 23 42;  
  --muted: 71 85 105;  
  --border: 226 232 240;  
  --accent: 37 99 235;  
  --accent-2: 14 165 233;  
  --danger: 220 38 38;  
  --focus: 2 132 199;  
  --base-font-size: 16px;  
}  
  
:root[data-contrast="high"] {  
  --bg: 255 255 255;  
  --surface: 255 255 255;  
  --surface-2: 243 244 246;  
  --fg: 0 0 0;  
  --muted: 17 24 39;  
  --border: 17 24 39;  
  --accent: 0 0 0;  
  --accent-2: 0 0 0;  
  --focus: 234 179 8;  
}  
  
:root[data-text-scale="sm"] {  
  --base-font-size: 15px;  
}  
  
:root[data-text-scale="base"] {  
  --base-font-size: 16px;  
}  
  
:root[data-text-scale="lg"] {  
  --base-font-size: 18px;  
}  
  
html {
```

```
    font-size: var(--base-font-size);
    scroll-behavior: smooth;
}

body {
    background: rgb(var(--bg));
    color: rgb(var(--fg));
    min-height: 100vh;
}

* {
    border-color: rgb(var(--border));
}

a,
button,
input,
select,
textarea,
summary {
    transition: 160ms ease;
}

a:focus-visible,
button:focus-visible,
input:focus-visible,
select:focus-visible,
textarea:focus-visible,
summary:focus-visible {
    outline: 3px solid rgb(var(--focus));
    outline-offset: 2px;
}

summary {
    list-style: none;
}

summary::-webkit-details-marker {
    display: none;
}

.container-shell {
    @apply mx-auto max-w-7xl px-4 sm:px-6 lg:px-8;
}

.card {
    background: rgb(var(--surface));
    border: 1px solid rgb(var(--border));
    @apply rounded-3xl shadow-sm;
}

.soft-card {
```

```
background: rgb(var(--surface-2));
border: 1px solid rgb(var(--border));
@apply rounded-2xl;
}

.badge {
  @apply inline-flex items-center rounded-full px-3 py-1 text-xs font-semibold;
  background: rgb(var(--surface-2));
  color: rgb(var(--fg));
}

.btn-primary {
  background: rgb(var(--accent));
  color: white;
  @apply inline-flex items-center justify-center rounded-2xl px-4 py-2 font-semib
}

.btn-primary:hover {
  transform: translateY(-1px);
}

.btn-secondary {
  background: rgb(var(--surface));
  color: rgb(var(--fg));
  border: 1px solid rgb(var(--border));
  @apply inline-flex items-center justify-center rounded-2xl px-4 py-2 font-semib
}

.input {
  width: 100%;
  background: rgb(var(--surface));
  color: rgb(var(--fg));
  border: 1px solid rgb(var(--border));
  @apply rounded-2xl px-4 py-3;
}

.label {
  @apply mb-2 block text-sm font-medium;
}

.page-title {
  @apply text-3xl font-bold tracking-tight sm:text-4xl;
}

.page-subtitle {
  color: rgb(var(--muted));
}

</> CSS
```



src/app/layout.tsx

```
import type { Metadata } from "next";
import "./globals.css";
import { createClient } from "@lib/supabase/server";
import { AccessibilityProvider } from "@components/accessibility-provider";
import { SiteHeader } from "@components/site-header";
import type { AccessibilityPreferences } from "@lib/types";

export const metadata: Metadata = {
  title: "Campus Companion",
  description: "Accessible student companion for first-year students",
};

const defaultPreferences: AccessibilityPreferences = {
  high_contrast: false,
  text_scale: "base",
};

export default async function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  let initialPreferences = defaultPreferences;

  if (user) {
    const { data } = await supabase
      .from("accessibility_preferences")
      .select("high_contrast, text_scale")
      .eq("user_id", user.id)
      .maybeSingle();

    if (data) {
      initialPreferences = data as AccessibilityPreferences;
    }
  }

  return (
    <html lang="en">
      <body>
        <AccessibilityProvider
          initialPreferences={initialPreferences}
          userId={user?.id ?? null}
        >
          <div className="min-h-screen">
```

```
    <SiteHeader />
    <main className="container-shell py-8">{children}</main>
  </div>
</AccessibilityProvider>
</body>
</html>
```

</> TypeScript



src/app/page.tsx

```
import Link from "next/link";
import {
  CalendarDays,
  MapPinned,
  PartyPopper,
  ShieldQuestion,
  Users,
} from "lucide-react";

const cards = [
  {
    href: "/timetable",
    title: "Timetable",
    text: "Weekly academic timetable with group selection and week navigation.",
    icon: CalendarDays,
  },
  {
    href: "/events",
    title: "Events",
    text: "Browse campus events, view details, and save your interest.",
    icon: PartyPopper,
  },
  {
    href: "/societies",
    title: "Society Events",
    text: "Join one or more societies and receive updates about their events.",
    icon: Users,
  },
  {
    href: "/map",
    title: "Map",
    text: "Open campus map with clear tags and high contrast location markers.",
    icon: MapPinned,
  },
  {
    href: "/student-support",
    title: "Lost and Found + Helpdesk",
    text: "Submit support messages and lost or found items through one page.",
  },
];
```

```

    icon: ShieldQuestion,
  },
];

export default function HomePage() {
  return (
    <div className="space-y-10">
      <section className="card overflow-hidden p-8 sm:p-12">
        <div className="grid gap-8 lg:grid-cols-[1.3fr_0.7fr]">
          <div>
            <span className="badge">Campus Companion</span>
            <h1 className="mt-5 text-4xl font-bold tracking-tight sm:text-5xl">
              A modern first-year campus app built for clarity and accessibility.
            </h1>
            <p className="page-subtitle">
              Students can sign in, manage their profile, save a timetable
              group, mark interest in events, join societies, receive
              notifications, submit support requests, and use accessibility
              settings from the top-right gear icon.
            </p>
            <div className="mt-6 flex flex-wrap gap-3">
              <Link href="/auth" className="btn-primary">
                Sign up or log in
              </Link>
              <Link href="/events" className="btn-secondary">
                Explore events
              </Link>
            </div>
          </div>

          <div className="soft-card p-6">
            <h2 className="text-lg font-semibold">What this starter includes</h2>
            <ul className="mt-4 space-y-3 text-sm leading-6 text-slate-600">
              <li>Accessible account menu and gear settings</li>
              <li>Supabase auth, profiles, RLS, and saved preferences</li>
              <li>Separate pages for timetable, events, societies, and map</li>
              <li>Shared support page with a sidebar switcher</li>
              <li>Notification bell with unread red dot for signed-in users</li>
            </ul>
          </div>
        </div>
      </section>

      <section className="grid gap-5 md:grid-cols-2 xl:grid-cols-3">
        {cards.map(({ href, title, text, icon: Icon }) => (
          <Link
            key={href}
            href={href}
            className="card group p-6 hover:-translate-y-1"
          >
            <div className="flex items-start justify-between gap-4">
              <div>

```



```

        <h2 className="text-xl font-semibold">{title}</h2>
        <p className="mt-2 text-sm leading-6 text-slate-600">{text}</p>
      </div>
      <span className="rounded-2xl bg-slate-100 p-3">
        <Icon className="h-5 w-5" />
      </span>
    </div>
  </Link>
  )))
</section>
</div>

```

</> TypeScript



src/components/accessibility-provider.tsx

```

"use client";

import {
  createContext,
  useContext,
  useEffect,
  useMemo,
  useState,
  type ReactNode,
} from "react";
import { createClient } from "@lib/supabase/client";
import type { AccessibilityPreferences, TextScale } from "@lib/types";

type AccessibilityContextValue = {
  preferences: AccessibilityPreferences;
  setHighContrast: (value: boolean) => void;
  setTextScale: (value: TextScale) => void;
};

const AccessibilityContext = createContext<
  AccessibilityContextValue | undefined
>(undefined);

export function AccessibilityProvider({
  children,
  initialPreferences,
  userId,
}: {
  children: ReactNode;
  initialPreferences: AccessibilityPreferences;
  userId: string | null;
}) {

```

```
const [preferences, setPreferences] =
  useState<AccessibilityPreferences>(initialPreferences);

const supabase = useMemo(() => createClient(), []);

useEffect(() => {
  const saved = window.localStorage.getItem("cc_accessibility");
  if (!saved) return;

  try {
    const parsed = JSON.parse(saved) as AccessibilityPreferences;
    setPreferences(parsed);
  } catch {}
}, []);

useEffect(() => {
  document.documentElement.dataset.contrast = preferences.high_contrast
    ? "high"
    : "default";
  document.documentElement.dataset.textScale = preferences.text_scale;
  window.localStorage.setItem(
    "cc_accessibility",
    JSON.stringify(preferences),
  );

  if (!userId) return;

  const save = async () => {
    await supabase.from("accessibility_preferences").upsert(
      {
        user_id: userId,
        high_contrast: preferences.high_contrast,
        text_scale: preferences.text_scale,
      },
      { onConflict: "user_id" },
    );
  };

  void save();
}, [preferences, userId, supabase]);

return (
  <AccessibilityContext.Provider
    value={{
      preferences,
      setHighContrast: (value) =>
        setPreferences((current) => ({ ...current, high_contrast: value })),
      setTextScale: (value) =>
        setPreferences((current) => ({ ...current, text_scale: value })),
    }}
  >
    {children}
  </AccessibilityContext.Provider>
  </div>
```

```

    </AccessibilityContext.Provider>
  );
}

export function useAccessibility() {
  const context = useContext(AccessibilityContext);

  if (!context) {
    throw new Error("useAccessibility must be used inside AccessibilityProvider")
  }
}

```

</> TypeScript



src/components/accessibility-panel.tsx

```

"use client";

import { Settings2 } from "lucide-react";
import { useAccessibility } from "@/components/accessibility-provider";

export function AccessibilityPanel() {
  const { preferences, setHighContrast, setTextScale } = useAccessibility();

  return (
    <details className="relative">
      <summary
        className="btn-secondary cursor-pointer px-3 py-2"
        aria-label="Accessibility settings"
      >
        <Settings2 className="h-5 w-5" />
      </summary>

      <div className="card absolute right-0 z-40 mt-3 w-80 p-5">
        <h2 className="text-lg font-semibold">Accessibility settings</h2>

        <div className="mt-5 space-y-5">
          <label className="flex items-center justify-between gap-4">
            <span className="text-sm font-medium">High contrast mode</span>
            <input
              type="checkbox"
              checked={preferences.high_contrast}
              onChange={(e) => setHighContrast(e.target.checked)}
              className="h-5 w-5"
            />
          </label>

          <div>

```

```

<label htmlFor="text-scale" className="label">
  Text size
</label>
<select
  id="text-scale"
  className="input"
  value={preferences.text_scale}
  onChange={(e) =>
    setTextScale(e.target.value as "sm" | "base" | "lg")
  }
>
  <option value="sm">Smaller</option>
  <option value="base">Default</option>
  <option value="lg">Larger</option>
</select>
</div>
</div>
</div>
</details>

```

</> TypeScript



src/components/account-menu.tsx

```

"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";

export function AccountMenu() {
  const router = useRouter();

  const handleSignOut = async () => {
    const supabase = createClient();
    await supabase.auth.signOut();
    router.push("/");
    router.refresh();
  };

  return (
    <details className="relative">
      <summary className="btn-secondary cursor-pointer">My Account</summary>

      <div className="card absolute right-0 z-40 mt-3 w-56 p-2">
        <Link href="/account" className="block rounded-2xl px-4 py-3 hover:bg-sla">
          My account
        </Link>
        <Link

```

```

      href="/auth"
      className="block rounded-2xl px-4 py-3 hover:bg-slate-50"
    >
      Forgot password
    </Link>
    <button
      type="button"
      onClick={handleSignOut}
      className="w-full rounded-2xl px-4 py-3 text-left hover:bg-slate-50"
    >
      Sign out
    </button>
  </div>
</details>
):

```

</> TypeScript



src/components/notification-bell.tsx

```

"use client";

import { useEffect, useMemo, useState } from "react";
import Link from "next/link";
import { Bell } from "lucide-react";
import { createClient } from "@/lib/supabase/client";

export function NotificationBell({
  userId,
  initialUnreadCount,
}): {
  userId: string;
  initialUnreadCount: number;
}) {
  const supabase = useMemo(() => createClient(), []);
  const [unreadCount, setUnreadCount] = useState(initialUnreadCount);

  useEffect(() => {
    const refreshCount = async () => {
      const { count } = await supabase
        .from("notifications")
        .select("*", { count: "exact", head: true })
        .eq("user_id", userId)
        .eq("is_read", false);

      setUnreadCount(count ?? 0);
    };

    const channel = supabase

```

```

    .channel(`notifications-${userId}`)
    .on(
      "postgres_changes",
      {
        event: "*",
        schema: "public",
        table: "notifications",
        filter: `user_id=eq.${userId}`,
      },
      () => {
        void refreshCount();
      },
    )
    .subscribe();

    return () => {
      void supabase.removeChannel(channel);
    };
  }, [supabase, userId]);

  return (
    <Link
      href="/account#notifications"
      className="relative rounded-2xl border border-slate-200 bg-white p-2"
      aria-label="Notifications"
    >
      <Bell className="h-5 w-5" />
      {unreadCount > 0 ? (
        <span
          className="absolute right-1 top-1 h-2.5 w-2.5 rounded-full bg-red-600"
          aria-hidden="true"
        />
      ) : null}
    </Link>
  );
}

```

</> TypeScript



src/components/notifications-list.tsx

```

"use client";

import { useMemo, useState } from "react";
import { createClient } from "@lib/supabase/client";

type NotificationItem = {
  id: string;
  title: string;
  body: string;
}

```

```

    href: string | null;
    is_read: boolean;
    created_at: string;
  };

export function NotificationsList({
  userId,
  initialNotifications,
}: {
  userId: string;
  initialNotifications: NotificationItem[];
}) {
  const [notifications, setNotifications] =
    useState<NotificationItem[]>(initialNotifications);
  const supabase = useMemo(() => createClient(), []);

  const markAsRead = async (id: string) => {
    await supabase
      .from("notifications")
      .update({ is_read: true })
      .eq("id", id)
      .eq("user_id", userId);

    setNotifications((current) =>
      current.map((item) =>
        item.id === id ? { ...item, is_read: true } : item,
      ),
    );
  };

  const markAllAsRead = async () => {
    const unreadIds = notifications.filter((item) => !item.is_read).map((item) =>
      item.id
    );
    if (!unreadIds.length) return;

    await supabase
      .from("notifications")
      .update({ is_read: true })
      .in("id", unreadIds)
      .eq("user_id", userId);

    setNotifications((current) =>
      current.map((item) => ({ ...item, is_read: true })),
    );
  };

  return (
    <div className="card p-6" id="notifications">
      <div className="flex flex-wrap items-center justify-between gap-3">
        <h2 className="text-xl font-semibold">Notifications</h2>
        <button type="button" onClick={markAllAsRead} className="btn-secondary">
          Mark all as read
        </button>
      </div>
    </div>
  );
}

```

```

</div>

<div className="mt-5 space-y-4">
  {notifications.length === 0 ? (
    <p className="text-sm text-slate-600">No notifications yet.</p>
  ) : (
    notifications.map((item) => (
      <div
        key={item.id}
        className="soft-card flex flex-col gap-3 p-4 sm:flex-row sm:items-s
      >
      <div>
        <div className="flex items-center gap-2">
          <h3 className="font-semibold">{item.title}</h3>
          {!item.is_read ? (
            <span className="rounded-full bg-red-600 px-2 py-0.5 text-xs
              New
            </span>
          ) : null}
        </div>
        <p className="mt-1 text-sm text-slate-600">{item.body}</p>
        {item.href ? (
          <a href={item.href} className="mt-2 inline-block text-sm font-m
            Open
          </a>
        ) : null}
      </div>

      {!item.is_read ? (
        <button
          type="button"
          onClick={() => markAsRead(item.id)}
          className="btn-secondary"
        >
          Mark read
        </button>
      ) : null}
    </div>
  ))
}</div>
</div>

```

 TypeScript


src/components/site-header.tsx


```

import Link from "next/link";
import { createClient } from "@lib/supabase/server";
import { AccessibilityPanel } from "@components/accessibility-panel";
import { AccountMenu } from "@components/account-menu";
import { NotificationBell } from "@components/notification-bell";

const links = [
  { href: "/timetable", label: "Timetable" },
  { href: "/events", label: "Events" },
  { href: "/societies", label: "Society Events" },
  { href: "/map", label: "Map" },
  { href: "/student-support", label: "Support" },
];

export async function SiteHeader() {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  let unreadCount = 0;

  if (user) {
    const { count } = await supabase
      .from("notifications")
      .select("*", { count: "exact", head: true })
      .eq("user_id", user.id)
      .eq("is_read", false);

    unreadCount = count ?? 0;
  }

  return (
    <header className="border-b border-slate-200 bg-white/85 backdrop-blur">
      <div className="container-shell flex min-h-20 flex-col gap-4 py-4 lg:flex-r">
        <div className="flex items-center gap-4">
          <Link href="/" className="text-xl font-bold tracking-tight">
            Campus Companion
          </Link>

          <nav aria-label="Primary navigation" className="hidden gap-2 lg:flex">
            {links.map((link) => (
              <Link
                key={link.href}
                href={link.href}
                className="rounded-2xl px-4 py-2 text-sm font-medium text-slate-7
              >
                {link.label}
              </Link>
            ))}
          </nav>
        </div>

```

```

<div className="flex flex-wrap items-center gap-3">
  {user ? <AccountMenu /> : <Link href="/auth" className="btn-secondary">
  {user ? (
    <NotificationBell userId={user.id} initialUnreadCount={unreadCount} /
  ) : null}
  <AccessibilityPanel />
</div>

<nav aria-label="Mobile navigation" className="flex flex-wrap gap-2 lg:hi
  {links.map((link) => (
    <Link
      key={link.href}
      href={link.href}
      className="rounded-2xl bg-slate-100 px-4 py-2 text-sm font-medium"
    >
      {link.label}
    </Link>
  ))}
</nav>
</div>

```

</> TypeScript



}

src/app/auth/page.tsx

```

"use client";

import { useMemo, useState } from "react";
import Link from "next/link";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";

export default function AuthPage() {
  const supabase = useMemo(() => createClient(), []);
  const router = useRouter();

  const [mode, setMode] = useState<"login" | "signup">("signup");
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

```

</> TypeScript



src/app/auth/update-password/page.tsx

```
"use client";

import { useMemo, useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@lib/supabase/client";

export default function UpdatePasswordPage() {
  const supabase = useMemo(() => createClient(), []);
  const router = useRouter();
  const [password, setPassword] = useState("");
  const [message, setMessage] = useState("");

  const onSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
  }
}
```

</> TypeScript



src/app/account/page.tsx

```
import { redirect } from "next/navigation";
import { createClient } from "@lib/supabase/server";
import { NotificationsList } from "@components/notifications-list";

export default async function AccountPage() {
  const supabase = await createClient();

  const { data: claimsData } = await supabase.auth.getClaims();
  const userId = claimsData?.claims?.sub as string | undefined;

  if (!userId) {
    redirect("/auth");
  }

  const {
    data: { user },
  } = await supabase.auth.getUser();

  const { data: profile } = await supabase
    .from("profiles")
    .select("full_name, email, selected_timetable_group_id")
    .eq("id", userId)
    .single();

  const { data: selectedGroup } = profile?.selected_timetable_group_id
    ? await supabase

```

```

        .from("timetable_groups")
        .select("code, title")
        .eq("id", profile.selected_timetable_group_id)
        .single()
      : { data: null };

const { data: memberships } = await supabase
  .from("society_memberships")
  .select("id, societies(name)")
  .eq("user_id", userId);

const { data: notifications } = await supabase
  .from("notifications")
  .select("id, title, body, href, is_read, created_at")
  .eq("user_id", userId)
  .order("created_at", { ascending: false });

return (
  <div className="space-y-8">
    <section className="card p-8">
      <h1 className="page-title">My account</h1>
      <p className="page-subtitle">
        View your account details, joined societies, selected timetable group,
        and notifications.
      </p>
    </section>

    <section className="grid gap-6 lg:grid-cols-2">
      <div className="card p-6">
        <h2 className="text-xl font-semibold">Profile</h2>
        <dl className="mt-5 space-y-4">
          <div>
            <dt className="text-sm font-medium text-slate-500">Name</dt>
            <dd className="mt-1 font-medium">{profile?.full_name}</dd>
          </div>
          <div>
            <dt className="text-sm font-medium text-slate-500">Email</dt>
            <dd className="mt-1 font-medium">{user?.email ?? profile?.email}</dd>
          </div>
          <div>
            <dt className="text-sm font-medium text-slate-500">Selected timetable</dt>
            <dd className="mt-1 font-medium">
              {selectedGroup ? `${selectedGroup.code} · ${selectedGroup.title}` : ''}
            </dd>
          </div>
        </dl>
      </div>

      <div className="card p-6">
        <h2 className="text-xl font-semibold">Joined societies</h2>
        <div className="mt-5 flex flex-wrap gap-2">
          {memberships && memberships.length > 0 ? (

```

```

        memberships.map((membership: any) => (
          <span key={membership.id} className="badge">
            {membership.societies?.name ?? "Society"}
          </span>
        ))
      ) : (
        <p className="text-sm text-slate-600">You have not joined any socie
      )}
    </div>
  </div>
</section>

<NotificationsList
  userId={userId}
  initialNotifications={({notifications ?? []}) as any[]}
/>
</div>

```

</> TypeScript



src/components/timetable-view.tsx

```

"use client";

import { useMemo, useState } from "react";
import { ChevronLeft, ChevronRight } from "lucide-react";
import { createClient } from "@lib/supabase/client";
import type { TimetableEntry, TimetableGroup } from "@lib/types";

const weekdays = ["Mon", "Tue", "Wed", "Thu", "Fri"];
const hours = Array.from({ length: 9 }, (_, index) => index + 8);

function getMonday(date: Date) {
  const cloned = new Date(date);
  const day = cloned.getDay();
  const diff = day === 0 ? -6 : 1 - day;
  cloned.setDate(cloned.getDate() + diff);
  cloned.setHours(0, 0, 0, 0);
  return cloned;
}

function formatDay(date: Date) {
  return new Intl.DateTimeFormat("en-IE", {
    day: "numeric",
    month: "short",
  }).format(date);
}

```

```

export function TimetableView({
  groups,
  entries,
  initialGroupId,
  userId,
}: {
  groups: TimetableGroup[];
  entries: TimetableEntry[];
  initialGroupId: string;
  userId: string | null;
}) {
  const supabase = useMemo(() => createClient(), []);
  const [weekOffset, setWeekOffset] = useState(0);
  const [selectedGroupId, setSelectedGroupId] = useState(initialGroupId);
  const [saving, setSaving] = useState(false);

  const monday = getMonday(new Date());
  monday.setDate(monday.getDate() + weekOffset * 7);

  const days = weekdays.map((label, index) => {
    const date = new Date(monday);
    date.setDate(monday.getDate() + index);
    return {
      label,
      formatted: formatDay(date),
      dayOfWeek: index + 1,
    };
  });

  const visibleEntries = entries.filter((entry) => entry.group_id === selectedGro

  const savePreferredGroup = async (groupId: string) => {
    if (!userId) return;

    setSaving(true);
    await supabase
      .from("profiles")
      .update({ selected_timetable_group_id: groupId })
      .eq("id", userId);
    setSaving(false);
  };

  return (
    <div className="space-y-6">
      <div className="card p-6">
        <div className="flex flex-col gap-4 lg:flex-row lg:items-end lg:justify-b
          <div>
            <h1 className="page-title">Timetable</h1>
            <p className="page-subtitle">
              Week view from 08:00 to 17:00. Students can view all groups and
              save one preferred timetable group to their profile.
            </p>

```

```

</div>

<div className="grid gap-3 sm:grid-cols-[auto_auto]">
  <div>
    <label htmlFor="group-select" className="label">
      Timetable group
    </label>
    <select
      id="group-select"
      className="input"
      value={selectedGroupId}
      onChange={(e) => {
        const value = e.target.value;
        setSelectedGroupId(value);
        void savePreferredGroup(value);
      }}
    >
      {groups.map((group) => (
        <option key={group.id} value={group.id}>
          {group.code} · {group.title}
        </option>
      ))}
    </select>
  </div>

  <div className="flex items-end gap-2">
    <button
      type="button"
      onClick={() => setWeekOffset((current) => current - 1)}
      className="btn-secondary"
    >
      <ChevronLeft className="mr-2 h-4 w-4" />
      Previous week
    </button>
    <button
      type="button"
      onClick={() => setWeekOffset((current) => current + 1)}
      className="btn-secondary"
    >
      Next week
      <ChevronRight className="ml-2 h-4 w-4" />
    </button>
  </div>
</div>
</div>

{userId ? (
  <p className="mt-4 text-sm text-slate-600">
    {saving ? "Saving preferred group..." : "Preferred group is saved to"}
  </p>
) : (
  <p className="mt-4 text-sm text-slate-600">

```

Sign in to save one timetable group to your account.

```

    </p>
  })
</div>

<div className="card overflow-x-auto">
  <table className="min-w-full border-collapse">
    <caption className="sr-only">Weekly timetable</caption>
    <thead>
      <tr>
        <th className="border-b p-4 text-left">Time</th>
        {days.map((day) => (
          <th key={day.label} className="border-b p-4 text-left">
            <div className="font-semibold">{day.label}</div>
            <div className="text-sm text-slate-500">{day.formatted}</div>
          </th>
        ))}
      </tr>
    </thead>
    <tbody>
      {hours.map((hour) => (
        <tr key={hour} className="align-top">
          <th className="border-b p-4 text-left text-sm font-semibold">
            {String(hour).padStart(2, "0")}:00
          </th>

          {days.map((day) => {
            const cellEntries = visibleEntries.filter(
              (entry) =>
                entry.day_of_week === day.dayOfWeek &&
                Number(entry.start_time.slice(0, 2)) === hour,
            );

            return (
              <td key={` ${hour} - ${day.label}`} className="border-b p-3">
                {cellEntries.length === 0 ? (
                  <div className="min-h-24 rounded-2xl border border-dashed
                    Free
                  </div>
                ) : (
                  <div className="space-y-3">
                    {cellEntries.map((entry) => (
                      <article
                        key={entry.id}
                        className="min-h-24 rounded-2xl border border-blue-
                      >
                        <div className="text-sm font-semibold">
                          {entry.module_code}
                        </div>
                        <div className="mt-1 text-sm">{entry.module_name}</div>
                        <div className="mt-2 text-xs text-slate-600">
                          {entry.start_time.slice(0, 5)}-{entry.end_time.sl

```



```

        </div>
        <div className="text-xs text-slate-600">
          {entry.lecturer_name}
        </div>
        <div className="text-xs text-slate-600">
          {entry.room_name}
        </div>
      </article>
    )}
  </div>
  )}
</td>
);
}}
</tr>
)}}
</tbody>
</table>
</div>

```

</> TypeScript



src/app/timetable/page.tsx

```

import { createClient } from "@lib/supabase/server";
import { TimetableView } from "@components/timetable-view";
import type { TimetableEntry, TimetableGroup } from "@lib/types";

export default async function TimetablePage() {
  const supabase = await createClient();

  const { data: groups } = await supabase
    .from("timetable_groups")
    .select("id, code, title")
    .order("code");

  const { data: entries } = await supabase
    .from("timetable_entries")
    .select(
      "id, group_id, day_of_week, start_time, end_time, module_code, module_name,"
    )
    .order("day_of_week")
    .order("start_time");

  const {
    data: { user },
  } = await supabase.auth.getUser();

```

```

let initialGroupId = groups?.[0]?.id ?? "";

if (user) {
  const { data: profile } = await supabase
    .from("profiles")
    .select("selected_timetable_group_id")
    .eq("id", user.id)
    .maybeSingle();

  if (profile?.selected_timetable_group_id) {
    initialGroupId = profile.selected_timetable_group_id;
  }
}

return (
  <TimetableView
    groups={{(groups ?? []) as TimetableGroup[]}}
    entries={{(entries ?? []) as TimetableEntry[]}}
    initialGroupId={initialGroupId}
    userId={user?.id ?? null}
  />
)

```

<> TypeScript 

src/components/event-interest-button.tsx

```

"use client";

import { useMemo, useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";

export function EventInterestButton({
  eventId,
  userId,
  initialInterested,
}: {
  eventId: string;
  userId: string | null;
  initialInterested: boolean;
}) {
  const router = useRouter();
  const supabase = useMemo(() => createClient(), []);
  const [interested, setInterested] = useState(initialInterested);
  const [busy, setBusy] = useState(false);

  const onToggle = async () => {

```

```

    if (!userId) {
      router.push("/auth");
      return;
    }

    setBusy(true);

    if (interested) {
      await supabase
        .from("event_interests")
        .delete()
        .eq("event_id", eventId)
        .eq("user_id", userId);
      setInterested(false);
    } else {
      await supabase.from("event_interests").insert({
        event_id: eventId,
        user_id: userId,
      });
      setInterested(true);
    }

    setBusy(false);
    router.refresh();
  };

  return (
    <button
      type="button"
      onClick={onToggle}
      disabled={busy}
      aria-pressed={interested}
      className={interested ? "btn-primary" : "btn-secondary"}
    >
      {busy ? "Saving..." : interested ? "Interested" : "I'm interested"}
    </button>
  )

```

</> TypeScript



src/app/events/page.tsx

```

import Link from "next/link";
import { createClient } from "@lib/supabase/server";
import { EventInterestButton } from "@components/event-interest-button";

export default async function EventsPage() {
  const supabase = await createClient();

  const { data: events } = await supabase

```

```

    .from("events")
    .select("*")
    .order("start_at", { ascending: true });

const {
  data: { user },
} = await supabase.auth.getUser();

let interestedIds = new Set<string>();

if (user) {
  const { data: interests } = await supabase
    .from("event_interests")
    .select("event_id")
    .eq("user_id", user.id);

  interestedIds = new Set((interests ?? []).map((item) => item.event_id));
}

return (
  <div className="space-y-8">
    <section className="card p-8">
      <h1 className="page-title">Events</h1>
      <p className="page-subtitle">
        Browse campus events, open a full detail page, and save whether you
        are interested. New events can be added directly in Supabase.
      </p>
    </section>

    <section className="grid gap-6 lg:grid-cols-2 xl:grid-cols-3">
      {(events ?? []).map((event) => (
        <article key={event.id} className="card overflow-hidden">
          <img
            src={event.image_url}
            alt={event.name}
            className="h-56 w-full object-cover"
          />
          <div className="p-6">
            <span className="badge">{event.tag}</span>
            <h2 className="mt-4 text-xl font-semibold">{event.name}</h2>
            <p className="mt-2 text-sm text-slate-600">{event.short_description}
            <dl className="mt-4 space-y-2 text-sm text-slate-600">
              <div>
                <dt className="sr-only">Date and time</dt>
                <dd>
                  {new Date(event.start_at).toLocaleString("en-IE", {
                    dateStyle: "medium",
                    timeStyle: "short",
                  })}
                </dd>
              </div>
            </dl>
          </div>
        </article>
      ))}
    </section>
  </div>
)

```

```

        <dt className="sr-only">Location</dt>
        <dd>{event.location}</dd>
      </div>
    </dl>

    <div className="mt-6 flex flex-wrap gap-3">
      <Link href={` /events/${event.id}`} className="btn-secondary">
        View details
      </Link>
      <EventInterestButton
        eventId={event.id}
        userId={user?.id ?? null}
        initialInterested={interestedIds.has(event.id)}
      />
    </div>
  </div>
</article>
  )))
</section>
</div>

```

</> TypeScript



src/app/events/[id]/page.tsx

```

import { notFound } from "next/navigation";
import { createClient } from "@lib/supabase/server";
import { EventInterestButton } from "@components/event-interest-button";

export default async function EventDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = await createClient();

  const { data: event } = await supabase.from("events").select("*").eq("id", id).

  if (!event) {
    notFound();
  }

  const {
    data: { user },
  } = await supabase.auth.getUser();

  let interested = false;

```

```

if (user) {
  const { data } = await supabase
    .from("event_interests")
    .select("id")
    .eq("event_id", id)
    .eq("user_id", user.id)
    .maybeSingle();

  interested = Boolean(data);
}

const { count } = await supabase
  .from("event_interests")
  .select("*, { count: \"exact\", head: true })
  .eq("event_id", id);

return (
  <div className="space-y-8">
    <img
      src={event.image_url}
      alt={event.name}
      className="card h-[24rem] w-full object-cover"
    />

    <section className="card p-8">
      <div className="flex flex-wrap items-center gap-3">
        <span className="badge">{event.tag}</span>
        <span className="badge">{count ?? 0} interested</span>
      </div>

      <h1 className="mt-5 text-3xl font-bold">{event.name}</h1>

      <dl className="mt-5 space-y-2 text-slate-600">
        <div>
          {new Date(event.start_at).toLocaleString("en-IE", {
            dateStyle: "full",
            timeStyle: "short",
          })}
        </div>
        <div>{event.location}</div>
      </dl>

      <p className="mt-6 max-w-3xl leading-7 text-slate-700">{event.description}

      <div className="mt-8">
        <EventInterestButton
          eventId={event.id}
          userId={user?.id ?? null}
          initialInterested={interested}
        />
      </div>

```

```
</section>
</div>
```

```
</> TypeScript
```



src/components/society-membership-button.tsx

```
"use client";

import { useMemo, useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";

export function SocietyMembershipButton({
  societyId,
  userId,
  initialJoined,
}: {
  societyId: string;
  userId: string | null;
  initialJoined: boolean;
}) {
  const router = useRouter();
  const supabase = useMemo(() => createClient(), []);
  const [joined, setJoined] = useState(initialJoined);
  const [busy, setBusy] = useState(false);

  const onToggle = async () => {
    if (!userId) {
      router.push("/auth");
      return;
    }

    setBusy(true);

    if (joined) {
      await supabase
        .from("society_memberships")
        .delete()
        .eq("society_id", societyId)
        .eq("user_id", userId);
      setJoined(false);
    } else {
      await supabase.from("society_memberships").insert({
        society_id: societyId,
        user_id: userId,
      });
      setJoined(true);
    }
  }
}
```

```

    }

    setBusy(false);
    router.refresh();
  };

  return (
    <button
      type="button"
      onClick={onToggle}
      disabled={busy}
      aria-pressed={joined}
      className={joined ? "btn-primary" : "btn-secondary"}
    >
      {busy ? "Saving..." : joined ? "Joined" : "Join society"}
    </button>
  );

```

</> TypeScript



src/components/society-event-interest-button.tsx

```

"use client";

import { useMemo, useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@lib/supabase/client";

export function SocietyEventInterestButton({
  societyEventId,
  userId,
  initialInterested,
}: {
  societyEventId: string;
  userId: string | null;
  initialInterested: boolean;
}) {
  const router = useRouter();
  const supabase = useMemo(() => createClient(), []);
  const [interested, setInterested] = useState(initialInterested);
  const [busy, setBusy] = useState(false);

  const onToggle = async () => {
    if (!userId) {
      router.push("/auth");
      return;
    }

    setBusy(true);

```



```

    if (interested) {
      await supabase
        .from("society_event_interests")
        .delete()
        .eq("society_event_id", societyEventId)
        .eq("user_id", userId);
      setInterested(false);
    } else {
      await supabase.from("society_event_interests").insert({
        society_event_id: societyEventId,
        user_id: userId,
      });
      setInterested(true);
    }

    setBusy(false);
    router.refresh();
  };

  return (
    <button
      type="button"
      onClick={onToggle}
      disabled={busy}
      aria-pressed={interested}
      className={interested ? "btn-primary" : "btn-secondary"}
    >
      {busy ? "Saving..." : interested ? "Interested" : "Interested in event"}
    </button>
  );

```

</> TypeScript



src/app/societies/page.tsx

```

import { createClient } from "@lib/supabase/server";
import { SocietyMembershipButton } from "@components/society-membership-button";
import { SocietyEventInterestButton } from "@components/society-event-interest-b

export default async function SocietiesPage() {
  const supabase = await createClient();

  const { data: societies } = await supabase
    .from("societies")
    .select("*")
    .order("name");

  const { data: societyEvents } = await supabase
    .from("society_events")
    .select("*")

```

```

.order("start_at", { ascending: true });

const {
  data: { user },
} = await supabase.auth.getUser();

let joinedSocietyIds = new Set<string>();
let interestedSocietyEventIds = new Set<string>();

if (user) {
  const { data: memberships } = await supabase
    .from("society_memberships")
    .select("society_id")
    .eq("user_id", user.id);

  const { data: interests } = await supabase
    .from("society_event_interests")
    .select("society_event_id")
    .eq("user_id", user.id);

  joinedSocietyIds = new Set((memberships ?? []).map((item) => item.society_id))
  interestedSocietyEventIds = new Set(
    (interests ?? []).map((item) => item.society_event_id),
  );
}

return (
  <div className="space-y-8">
    <section className="card p-8">
      <h1 className="page-title">Society events</h1>
      <p className="page-subtitle">
        Students can join one or more societies, browse all society events,
        and receive notifications only for societies they joined.
      </p>
    </section>

    <section className="space-y-8">
      {(societies ?? []).map((society) => {
        const items = (societyEvents ?? []).filter(
          (event) => event.society_id === society.id,
        );

        return (
          <article key={society.id} className="card overflow-hidden">
            <div className="grid gap-0 lg:grid-cols-[0.9fr_1.1fr]">
              <img
                src={society.hero_image_url}
                alt={society.name}
                className="h-full min-h-72 w-full object-cover"
              />

              <div className="p-8">

```

```

<div className="flex flex-wrap items-center gap-3">
  <span className="badge">{society.tag}</span>
  <SocietyMembershipButton
    societyId={society.id}
    userId={user?.id ?? null}
    initialJoined={joinedSocietyIds.has(society.id)}
  />
</div>

<h2 className="mt-5 text-2xl font-semibold">{society.name}</h2>
<p className="mt-3 max-w-2xl text-slate-600">{society.description}</p>

<div className="mt-8 space-y-4">
  {items.map((event) => (
    <div key={event.id} className="soft-card p-5">
      <div className="flex flex-col gap-4 xl:flex-row xl:items-center">
        <div>
          <div className="flex flex-wrap items-center gap-2">
            <h3 className="text-lg font-semibold">{event.name}</h3>
            <span className="badge">{event.tag}</span>
          </div>
          <p className="mt-2 text-sm leading-6 text-slate-600">
            {event.description}
          </p>
          <div className="mt-3 text-sm text-slate-600">
            {new Date(event.start_at).toLocaleString("en-IE", {
              dateStyle: "medium",
              timeStyle: "short",
            })}{event.location}
          </div>
        </div>
        <div className="flex gap-3">
          <SocietyEventInterestButton
            societyEventId={event.id}
            userId={user?.id ?? null}
            initialInterested={interestedSocietyEventIds.has(event.id)}
          />
        </div>
      </div>
    </div>
  ))}

  {items.length === 0 ? (
    <p className="text-sm text-slate-600">
      No events added for this society yet.
    </p>
  ) : null}
</div>
</div>
</div>

```

```

        </article>
      );
    }
  }
</section>
</div>

```

⌘ TypeScript



src/components/campus-map.tsx

```

"use client";

import dynamic from "next/dynamic";
import type { MapLocation } from "@lib/types";

const CampusMapInner = dynamic(() => import("@/components/campus-map-inner"), {
  ssr: false,
  loading: () => (
    <div className="card flex h-[34rem] items-center justify-center">
      <p className="text-slate-600">Loading map...</p>
    </div>
  ),
});

export function CampusMap({ locations }: { locations: MapLocation[] }) {
  return <CampusMapInner locations={locations} />;
}

```

⌘ TypeScript



src/components/campus-map-inner.tsx

```

"use client";

import { MapContainer, Marker, Popup, TileLayer } from "react-leaflet";
import L from "leaflet";
import type { MapLocation } from "@lib/types";

const markerIcon = L.divIcon({
  className: "",
  html: `
    <div
      style="
        width:20px;
        height:20px;
        border-radius:9999px;

```

```

        background:#ef4444;
        border:3px solid #ffffff;
        box-shadow:0 0 0 3px #111827;
    "
    ></div>
    `
    ,
    iconSize: [20, 20],
    iconAnchor: [10, 10],
  });

export default function CampusMapInner({
  locations,
}): {
  locations: MapLocation[];
}) {
  const first = locations[0];

  return (
    <div className="grid gap-6 lg:grid-cols-[1.2fr_0.8fr]">
      <div className="card overflow-hidden">
        <MapContainer
          center={[
            Number(first?.latitude ?? 53.349805),
            Number(first?.longitude ?? -6.26031),
          ]}
          zoom={16}
          style={{ height: "34rem", width: "100%" }}
          scrollWheelZoom
        >
          <TileLayer
            attribution='&copy; OpenStreetMap contributors'
            url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
          />

          {locations.map((location) => (
            <Marker
              key={location.id}
              position={[Number(location.latitude), Number(location.longitude)]}
              icon={markerIcon}
            >
              <Popup>
                <div className="space-y-2">
                  <div className="font-semibold">{location.name}</div>
                  <div className="text-sm">{location.tag}</div>
                  {location.description ? (
                    <div className="text-sm">{location.description}</div>
                  ) : null}
                </div>
              </Popup>
            </Marker>
          ))}
        </MapContainer>

```

```

</div>

<aside className="space-y-4">
  {locations.map((location) => (
    <div key={location.id} className="card p-5">
      <div className="flex items-center gap-2">
        <span className="inline-block h-3 w-3 rounded-full bg-red-500" />
        <h2 className="font-semibold">{location.name}</h2>
      </div>
      <p className="mt-2 text-sm font-medium text-slate-700">{location.tag}</p>
      <p className="mt-2 text-sm text-slate-600">{location.description}</p>
    </div>
  ))}
</aside>
</div>

```

</> TypeScript



src/app/map/page.tsx

```

import { createClient } from "@lib/supabase/server";
import { CampusMap } from "@components/campus-map";
import type { MapLocation } from "@lib/types";

export default async function MapPage() {
  const supabase = await createClient();

  const { data: locations } = await supabase
    .from("map_locations")
    .select("id, name, tag, latitude, longitude, description")
    .order("name");

  return (
    <div className="space-y-8">
      <section className="card p-8">
        <h1 className="page-title">Campus map</h1>
        <p className="page-subtitle">
          Open map with clear tags and visually contrasting markers. Add more
          map points directly in Supabase later.
        </p>
      </section>

      <CampusMap locations={((locations ?? []) as MapLocation[])} />
    </div>
  );
}

```

</> TypeScript



src/components/support-client.tsx

```
"use client";

import { useMemo, useState } from "react";
import { useRouter } from "next/navigation";
import { createClient } from "@/lib/supabase/client";

type TabKey = "lost-found" | "helpdesk";

export function SupportClient({
  userId,
  email,
}: {
  userId: string | null;
  email: string | null;
}) {
  const router = useRouter();
  const supabase = useMemo(() => createClient(), []);
  const [tab, setTab] = useState<TabKey>("lost-found");
  const [message, setMessage] = useState("");

  const submitLostFound = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();

    if (!userId) {
      router.push("/auth");
      return;
    }

    const form = new FormData(event.currentTarget);

    const { error } = await supabase.from("lost_found_items").insert({
      user_id: userId,
      item_type: form.get("itemType"),
      title: form.get("title"),
      description: form.get("description"),
      location: form.get("location"),
      contact_email: form.get("contactEmail"),
    });

    setMessage(error ? error.message : "Lost and found item submitted.");
    if (!error) event.currentTarget.reset();
  };

  const submitHelpdesk = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();

    if (!userId) {
```

```

    router.push("/auth");
    return;
  }

  const form = new FormData(event.currentTarget);

  const { error } = await supabase.from("helpdesk_messages").insert({
    user_id: userId,
    subject: form.get("subject"),
    category: form.get("category"),
    message: form.get("message"),
  });

  setMessage(error ? error.message : "Helpdesk message submitted.");
  if (!error) event.currentTarget.reset();
};

return (
  <div className="grid gap-6 lg:grid-cols-[0.28fr_0.72fr]">
    <aside className="card p-4">
      <button
        type="button"
        onClick={() => setTab("lost-found")}
        className={`mb-2 w-full rounded-2xl px-4 py-3 text-left font-medium ${
          tab === "lost-found" ? "bg-slate-900 text-white" : "bg-slate-100"
        }}`>
        >
        Lost and Found
      </button>

      <button
        type="button"
        onClick={() => setTab("helpdesk")}
        className={`w-full rounded-2xl px-4 py-3 text-left font-medium ${
          tab === "helpdesk" ? "bg-slate-900 text-white" : "bg-slate-100"
        }}`>
        >
        Helpdesk
      </button>
    </aside>

    <section className="card p-8">
      {tab === "lost-found" ? (
        <>
          <h2 className="text-2xl font-semibold">Lost and Found</h2>
          <p className="mt-3 text-slate-600">
            Students can submit lost or found items. Records are stored in
            Supabase for admin follow-up.
          </p>

          <form onSubmit={submitLostFound} className="mt-6 space-y-5">
            <div>

```



```
<label htmlFor="itemType" className="label">
  Item type
</label>
<select id="itemType" name="itemType" className="input" required>
  <option value="lost">Lost</option>
  <option value="found">Found</option>
</select>
</div>

<div>
  <label htmlFor="title" className="label">
    Item title
  </label>
  <input id="title" name="title" className="input" required />
</div>

<div>
  <label htmlFor="description" className="label">
    Description
  </label>
  <textarea
    id="description"
    name="description"
    className="input min-h-32"
    required
  />
</div>

<div>
  <label htmlFor="location" className="label">
    Location
  </label>
  <input id="location" name="location" className="input" required />
</div>

<div>
  <label htmlFor="contactEmail" className="label">
    Contact email
  </label>
  <input
    id="contactEmail"
    name="contactEmail"
    type="email"
    defaultValue={email ?? ""}
    className="input"
    required
  />
</div>

<button className="btn-primary" type="submit">
  Submit item
</button>
```

```

    </form>
  </>
) : (
  <>
    <h2 className="text-2xl font-semibold">Helpdesk</h2>
    <p className="mt-3 text-slate-600">
      Students can send help messages that go straight into Supabase for
      staff review.
    </p>

    <form onSubmit={submitHelpdesk} className="mt-6 space-y-5">
      <div>
        <label htmlFor="subject" className="label">
          Subject
        </label>
        <input id="subject" name="subject" className="input" required />
      </div>

      <div>
        <label htmlFor="category" className="label">
          Category
        </label>
        <select id="category" name="category" className="input" required>
          <option value="General">General</option>
          <option value="Technical">Technical</option>
          <option value="Accessibility">Accessibility</option>
          <option value="Timetable">Timetable</option>
        </select>
      </div>

      <div>
        <label htmlFor="message" className="label">
          Message
        </label>
        <textarea
          id="message"
          name="message"
          className="input min-h-40"
          required
        />
      </div>

      <button className="btn-primary" type="submit">
        Send message
      </button>
    </form>
  </>
)}

{message ? (
  <p className="mt-6 rounded-2xl bg-slate-100 px-4 py-3 text-sm">
    {message}
  </p>
) : (
  <p className="mt-6 rounded-2xl bg-slate-100 px-4 py-3 text-sm">
    No messages found.
  </p>
)}

```

```
    </p>
  ) : null}
</section>
```

❏ TypeScript



```
}
```

src/app/student-support/page.tsx

```
import { createClient } from "@lib/supabase/server";
import { SupportClient } from "@components/support-client";

export default async function StudentSupportPage() {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  return (
    <div className="space-y-8">
      <section className="card p-8">
        <h1 className="page-title">Student support</h1>
        <p className="page-subtitle">
          Lost and Found and Helpdesk live on one page with a sidebar switcher.
          Submitted records go into Supabase.
        </p>
      </section>

      <SupportClient
        userId={user?.id ?? null}
        email={user?.email ?? null}
      />
    </div>
  );
}
```

❏ TypeScript



3) Brief test steps

1. In VS Code terminal, run:

```
npm run dev
```



2. In Supabase:

- create a new project
- copy the Project URL and Publishable key into .env.local
- open SQL Editor and run supabase/schema.sql

3. Start the app and visit:

```
http://localhost:3000
```

4. Test auth:
 - sign up with name, email, password
 - after login, confirm Login / Sign up becomes My Account
 - confirm the bell icon appears only when logged in
5. Test timetable:
 - open /timetable
 - switch between G1, G2, G3
 - if signed in, refresh and confirm your selected group stayed saved
6. Test events:
 - open /events
 - click View details
 - toggle Interested
 - in Supabase, check event_interest_admin_view
7. Test societies:
 - join one or more societies
 - mark interest in a society event
 - in Supabase, check society_members_admin_view and society_event_intere
8. Test notifications:
 - while a user is a member of a society, insert a new row into society_e
 - refresh the app or wait for realtime update
 - confirm the bell shows a red dot
 - open My Account and mark notifications as read
9. Test support:
 - open /student-support
 - submit a Lost and Found item
 - submit a Helpdesk message
 - confirm both tables have records in Supabase
10. Test accessibility:
 - use the gear icon
 - toggle high contrast
 - change text size
 - refresh and confirm preferences still apply

For password reset redirects, Supabase requires the reset redirectTo URL to be allowed in Auth URL Configuration, and the change-password page should match that URL. Add your local and production update-password URLs there. [Supabase +1](#)

SELF-CHECK: likely accessibility issues and fixes

1. Dropdown menus built with <details>



 Risk:
 - usable, but not as strong as a full ARIA menu button pattern
 Fix:

- replace with a button + popover/dialog and manage focus on open/close

2. Color contrast on custom cards

Risk:

- future design edits may reduce contrast

Fix:

- test with Lighthouse and keep text/background contrast at WCAG AA or s

3. Map accessibility

Risk:

- interactive maps are harder for keyboard and screen-reader users

Fix:

- keep the side list of map locations, and ensure every marker has match

4. Table density on timetable

Risk:

- crowded cells on smaller screens

Fix:

- add a mobile day-by-day fallback view below 768px

5. Success and error feedback

Risk:

- screen readers may not announce state changes

Fix:

- add aria-live="polite" to confirmation and error message containers

6. Focus order

Risk:

- if more popovers are added later, focus can get lost

Fix:

- trap focus inside modal-style panels and return focus to the trigger o

7. Image alt text

Risk:

- remote event images might get weak alt text

Fix:

- keep alt text tied to the event or society name, not generic words lik

8. Bell red dot only

Risk:

- relying on color alone can miss some users

Fix:

- add visually hidden text such as "3 unread notifications"

Everything works except you are unable to click onto society events so when there are more than one you cant

click onto it, and view more details like you can with normal events, and you cant see the images for the society event

I found the gap: society events were only rendered as inline blocks, so there was no dedicated dynamic route to open them the way normal events do. I'm fixing it by adding a detail page for each society event and updating the society page cards to show images plus a "View details" link.

Thought for 59s >

Use a dynamic App Router segment for each society event and link each event card to it. Next.js supports dynamic segments like `[id]` for detail pages, and `Link` is the standard way to navigate to them. [Next.js +1](#)

Files to edit

```
src/app/societies/page.tsx
src/app/societies/events/[id]/page.tsx
```



Code only

src/app/societies/page.tsx

```
import Link from "next/link";
import { createClient } from "@lib/supabase/server";
import { SocietyMembershipButton } from "@components/society-membership-button";
import { SocietyEventInterestButton } from "@components/society-event-interest-b";

export default async function SocietiesPage() {
  const supabase = await createClient();

  const { data: societies } = await supabase
    .from("societies")
    .select("*")
    .order("name");

  const { data: societyEvents } = await supabase
    .from("society_events")
    .select("*")
    .order("start_at", { ascending: true });

  const {
    data: { user },
```

```

} = await supabase.auth.getUser();

let joinedSocietyIds = new Set<string>();
let interestedSocietyEventIds = new Set<string>();

if (user) {
  const { data: memberships } = await supabase
    .from("society_memberships")
    .select("society_id")
    .eq("user_id", user.id);

  const { data: interests } = await supabase
    .from("society_event_interests")
    .select("society_event_id")
    .eq("user_id", user.id);

  joinedSocietyIds = new Set((memberships ?? []).map((item) => item.society_id))
  interestedSocietyEventIds = new Set(
    (interests ?? []).map((item) => item.society_event_id),
  );
}

return (
  <div className="space-y-8">
    <section className="card p-8">
      <h1 className="page-title">Society events</h1>
      <p className="page-subtitle">
        Students can join one or more societies, browse all society events,
        and receive notifications only for societies they joined.
      </p>
    </section>

    <section className="space-y-8">
      {(societies ?? []).map((society) => {
        const items = (societyEvents ?? []).filter(
          (event) => event.society_id === society.id,
        );

        return (
          <article key={society.id} className="card overflow-hidden">
            <div className="grid gap-0 lg:grid-cols-[0.9fr_1.1fr]">
              <img
                src={society.hero_image_url}
                alt={society.name}
                className="h-full min-h-72 w-full object-cover"
              />

              <div className="p-8">
                <div className="flex flex-wrap items-center gap-3">
                  <span className="badge">{society.tag}</span>
                  <SocietyMembershipButton
                    societyId={society.id}

```

```

        userId={user?.id ?? null}
        initialJoined={joinedSocietyIds.has(society.id)}
      />
    </div>

    <h2 className="mt-5 text-2xl font-semibold">{society.name}</h2>
    <p className="mt-3 max-w-2xl text-slate-600">
      {society.description}
    </p>

    <div className="mt-8 space-y-4">
      {items.map((event) => (
        <article
          key={event.id}
          className="soft-card overflow-hidden"
        >
          <div className="grid gap-0 md:grid-cols-[220px_1fr]">
            <img
              src={event.image_url}
              alt={event.name}
              className="h-full min-h-52 w-full object-cover"
            />

            <div className="p-5">
              <div className="flex flex-col gap-4 xl:flex-row xl:it
                <div>
                  <div className="flex flex-wrap items-center gap-2"
                    <h3 className="text-lg font-semibold">
                      {event.name}
                    </h3>
                    <span className="badge">{event.tag}</span>
                  </div>

                  <p className="mt-2 text-sm leading-6 text-slate-600">
                    {event.description.length > 140
                      ? `${event.description.slice(0, 140)}...`
                      : event.description}
                  </p>

                  <div className="mt-3 text-sm text-slate-600">
                    {new Date(event.start_at).toLocaleString(
                      "en-IE",
                      {
                        dateStyle: "medium",
                        timeStyle: "short",
                      },
                    )}{ " "}
                    · {event.location}
                  </div>
                </div>
              </div>

              <div className="flex flex-wrap gap-3">

```



```

        <Link
            href={` /societies/events/${event.id}`}
            className="btn-secondary"
        >
            View details
        </Link>

        <SocietyEventInterestButton
            societyEventId={event.id}
            userId={user?.id ?? null}
            initialInterested={interestedSocietyEventIds.has(
                event.id,
            )}
        />
    </div>
</div>
</div>
</div>
</article>
)))

{items.length === 0 ? (
    <p className="text-sm text-slate-600">
        No events added for this society yet.
    </p>
    ) : null}
</div>
</div>
</div>
</article>

);
}}
</section>
</div>

```

⌘ TypeScript



```
src/app/societies/events/[id]/page.tsx
```

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { createClient } from "@lib/supabase/server";
import { SocietyEventInterestButton } from "@components/society-event-interest-b

export default async function SocietyEventDetailPage({
  params,
}): {
  params: Promise<{ id: string }>;
```

```
  }) {  
    const { id } = await params;  
    const supabase = await createClient();  
  
    const { data: event } = await supabase  
      .from("society_events")  
      .select(  
        `,  
        *,  
        societies (  
          id,  
          name,  
          tag  
        )  
      `,  
    )  
      .eq("id", id)  
      .single();  
  
    if (!event) {  
      notFound();  
    }  
  
    const {  
      data: { user },  
    } = await supabase.auth.getUser();  
  
    let interested = false;  
  
    if (user) {  
      const { data } = await supabase  
        .from("society_event_interests")  
        .select("id")  
        .eq("society_event_id", id)  
        .eq("user_id", user.id)  
        .maybeSingle();  
  
      interested = Boolean(data);  
    }  
  
    const { count } = await supabase  
      .from("society_event_interests")  
      .select("*, { count: \"exact\", head: true }")  
      .eq("society_event_id", id);  
  
    const societyName =  
      event.societies && !Array.isArray(event.societies)  
        ? event.societies.name  
        : "Society";  
  
    return (  
      <div className="space-y-8">
```

```

<Link href="/societies" className="btn-secondary">
  Back to societies
</Link>

<img
  src={event.image_url}
  alt={event.name}
  className="card h-[24rem] w-full object-cover"
/>

<section className="card p-8">
  <div className="flex flex-wrap items-center gap-3">
    <span className="badge">{event.tag}</span>
    <span className="badge">{societyName}</span>
    <span className="badge">{count ?? 0} interested</span>
  </div>

  <h1 className="mt-5 text-3xl font-bold">{event.name}</h1>

  <dl className="mt-5 space-y-2 text-slate-600">
    <div>
      {new Date(event.start_at).toLocaleString("en-IE", {
        dateStyle: "full",
        timeStyle: "short",
      })}
    </div>
    <div>{event.location}</div>
  </dl>

  <p className="mt-6 max-w-3xl leading-7 text-slate-700">
    {event.description}
  </p>

  <div className="mt-8">
    <SocietyEventInterestButton
      societyEventId={event.id}
      userId={user?.id ?? null}
      initialInterested={interested}
    />
  </div>
</section>
</div>

```

</> TypeScript



Brief test steps

1. Save both files and run:
npm run dev



2. Open `/societies`
3. Confirm each society event now shows:
 - an image
 - a View details button
 - an Interested button
4. Click View details on multiple different society events
5. Confirm each one opens its own page:
 - `/societies/events/[id]`
6. Confirm the detail page shows:
 - event image
 - event name
 - date and time
 - location
 - full description
 - interested count
 - interested toggle button
7. Go back to `/societies` and check the other society events also open corre

SELF-CHECK: likely accessibility issues and fixes

1. The society event image is informative, so keep alt text as the event name 
2. The "View details" link text is clear, but if you want even better screen reader support, use `View details for {event.name}`
3. If event descriptions get very long in the list view, keep the shortened version
4. On the detail page, the back link is keyboard accessible already, but you can add a label
5. If you later make the whole card clickable, do not nest buttons inside a button

The core fix is that society events need their own dynamic detail route, because inline society cards alone do not create a navigable page. Next.js App Router handles that with a folder like `events/[id]`, and `Link` gives the expected client-side navigation. Next.js +1

     ...  Sources

for society events i need short_description and description which i can add to supabase, so i will need the updated pages aswell as any file changes needed

Thought for 20s >

Yes — for **society events** you should have both:

- `short_description` for the society events list page
- `description` for the full society event details page

Files to change

```
supabase/schema.sql  
src/app/societies/page.tsx  
src/app/societies/events/[id]/page.tsx
```



1) Database change for Supabase

If your table already exists, run this in the **Supabase SQL Editor**:

```
alter table public.society_events  
add column if not exists short_description text;  
  
update public.society_events  
set short_description = left(description, 140)  
where short_description is null or short_description = '';
```

SQL



If you want your `supabase/schema.sql` starter file updated too, replace the `society_events` table and seed insert with this version:

`supabase/schema.sql` updated table

```
create table public.society_events (  
  id uuid primary key default gen_random_uuid(),  
  society_id uuid not null references public.societies(id) on delete cascade,  
  name text not null,  
  image_url text not null,  
  start_at timestamptz not null,  
  end_at timestamptz not null,  
  location text not null,  
  tag text not null,  
  short_description text not null,  
  description text not null,
```

</> SQL



/,

supabase/schema.sql updated seed data

```
insert into public.society_events (  
  society_id,  
  name,  
  image_url,  
  start_at,  
  end_at,  
  location,  
  tag,  
  short_description,  
  description  
)  
values  
(  
  (select id from public.societies where name = 'Finance Society'),  
  'ETF Basics Night',  
  'https://images.unsplash.com/photo-1559526324-4b87b5e36e44?q=80&w=1200&auto=f  
now() + interval '5 days',  
now() + interval '5 days 2 hours',  
  'Room F12',  
  'Workshop',  
  'A beginner friendly introduction to ETFs, investing basics, and long term fi  
  'An introductory session on ETFs, long term investing, and financial literacy  
) ,  
(  
  (select id from public.societies where name = 'Debate Society'),  
  'Mini Debate Tournament',  
  'https://images.unsplash.com/photo-1517048676732-d65bc937f952?q=80&w=1200&aut  
now() + interval '8 days',  
now() + interval '8 days 3 hours',  
  'Lecture Theatre 2',  
  'Competition',  
  'Friendly team debates with quick rounds designed to build confidence and pub  
  'Students form quick teams and take part in friendly debate rounds with start  
) ,  
(  
  (select id from public.societies where name = 'Tech Makers'),  
  'Campus App Jam',  
  'https://images.unsplash.com/photo-1515879218367-8466d910aaa4?q=80&w=1200&aut  
now() + interval '12 days',  
now() + interval '12 days 3 hours',  
  'Innovation Lab',  
  'Build',  
  'A collaborative build session where students create useful digital ideas for
```

</> SQL



2) Updated society list page

src/app/societies/page.tsx

```
import Link from "next/link";
import { createClient } from "@/lib/supabase/server";
import { SocietyMembershipButton } from "@/components/society-membership-button";
import { SocietyEventInterestButton } from "@/components/society-event-interest-b";

export default async function SocietiesPage() {
  const supabase = await createClient();

  const { data: societies } = await supabase
    .from("societies")
    .select("*")
    .order("name");

  const { data: societyEvents } = await supabase
    .from("society_events")
    .select("*")
    .order("start_at", { ascending: true });

  const {
    data: { user },
  } = await supabase.auth.getUser();

  let joinedSocietyIds = new Set<string>();
  let interestedSocietyEventIds = new Set<string>();

  if (user) {
    const { data: memberships } = await supabase
      .from("society_memberships")
      .select("society_id")
      .eq("user_id", user.id);

    const { data: interests } = await supabase
      .from("society_event_interests")
      .select("society_event_id")
      .eq("user_id", user.id);

    joinedSocietyIds = new Set((memberships ?? []).map((item) => item.society_id))
    interestedSocietyEventIds = new Set(
      (interests ?? []).map((item) => item.society_event_id),
    )
  }
}
```

```

    );
  }

  return (
    <div className="space-y-8">
      <section className="card p-8">
        <h1 className="page-title">Society events</h1>
        <p className="page-subtitle">
          Students can join one or more societies, browse all society events,
          and receive notifications only for societies they joined.
        </p>
      </section>

      <section className="space-y-8">
        {(societies ?? []).map((society) => {
          const items = (societyEvents ?? []).filter(
            (event) => event.society_id === society.id,
          );

          return (
            <article key={society.id} className="card overflow-hidden">
              <div className="grid gap-0 lg:grid-cols-[0.9fr_1.1fr]">
                <img
                  src={society.hero_image_url}
                  alt={society.name}
                  className="h-full min-h-72 w-full object-cover"
                />

                <div className="p-8">
                  <div className="flex flex-wrap items-center gap-3">
                    <span className="badge">{society.tag}</span>
                    <SocietyMembershipButton
                      societyId={society.id}
                      userId={user?.id ?? null}
                      initialJoined={joinedSocietyIds.has(society.id)}
                    />
                  </div>

                  <h2 className="mt-5 text-2xl font-semibold">{society.name}</h2>
                  <p className="mt-3 max-w-2xl text-slate-600">
                    {society.description}
                  </p>

                  <div className="mt-8 space-y-4">
                    {items.map((event) => (
                      <article
                        key={event.id}
                        className="soft-card overflow-hidden"
                      >
                        <div className="grid gap-0 md:grid-cols-[220px_1fr]">
                          <img
                            src={event.image_url}

```



```

    alt={event.name}
    className="h-full min-h-52 w-full object-cover"
  />

  <div className="p-5">
    <div className="flex flex-col gap-4 xl:flex-row xl:it
      <div>
        <div className="flex flex-wrap items-center gap-2
          <h3 className="text-lg font-semibold">
            {event.name}
          </h3>
          <span className="badge">{event.tag}</span>
        </div>

        <p className="mt-2 text-sm leading-6 text-slate-6
          {event.short_description}
        </p>

        <div className="mt-3 text-sm text-slate-600">
          {new Date(event.start_at).toLocaleString(
            "en-IE",
            {
              dateStyle: "medium",
              timeStyle: "short",
            },
          )}{ " "}
          · {event.location}
        </div>
      </div>

    <div className="flex flex-wrap gap-3">
      <Link
        href={` /societies/events/${event.id}`}
        className="btn-secondary"
      >
        View details
      </Link>

      <SocietyEventInterestButton
        societyEventId={event.id}
        userId={user?.id ?? null}
        initialInterested={interestedSocietyEventIds.has
          event.id,
        }
      />
    </div>
  </div>
</div>
</article>
)}}

```

```

        {items.length === 0 ? (
          <p className="text-sm text-slate-600">
            No events added for this society yet.
          </p>
        ) : null}
      </div>
    </div>
  </div>
</article>
);
}}}
</section>
</div>

```

</> TypeScript



3) Updated society event details page

src/app/societies/events/[id]/page.tsx

```

import Link from "next/link";
import { notFound } from "next/navigation";
import { createClient } from "@lib/supabase/server";
import { SocietyEventInterestButton } from "@components/society-event-interest-b

export default async function SocietyEventDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = await createClient();

  const { data: event } = await supabase
    .from("society_events")
    .select(
      `
        *,
        societies (
          id,
          name,
          tag
        )
      `,
    )
    .eq("id", id)

```

```
.single());

if (!event) {
  notFound();
}

const {
  data: { user },
} = await supabase.auth.getUser();

let interested = false;

if (user) {
  const { data } = await supabase
    .from("society_event_interests")
    .select("id")
    .eq("society_event_id", id)
    .eq("user_id", user.id)
    .maybeSingle();

  interested = Boolean(data);
}

const { count } = await supabase
  .from("society_event_interests")
  .select("*, { count: 'exact', head: true }")
  .eq("society_event_id", id);

const societyName =
  event.societies && !Array.isArray(event.societies)
    ? event.societies.name
    : "Society";

return (
  <div className="space-y-8">
    <Link href="/societies" className="btn-secondary">
      Back to societies
    </Link>

    <img
      src={event.image_url}
      alt={event.name}
      className="card h-[24rem] w-full object-cover"
    />

    <section className="card p-8">
      <div className="flex flex-wrap items-center gap-3">
        <span className="badge">{event.tag}</span>
        <span className="badge">{societyName}</span>
        <span className="badge">{count ?? 0} interested</span>
      </div>
    </section>
  </div>
)
```

```
<h1 className="mt-5 text-3xl font-bold">{event.name}</h1>

<p className="mt-3 text-lg text-slate-600">{event.short_description}</p>

<dl className="mt-5 space-y-2 text-slate-600">
  <div>
    {new Date(event.start_at).toLocaleString("en-IE", {
      dateStyle: "full",
      timeStyle: "short",
    })}
  </div>
  <div>{event.location}</div>
</dl>

<p className="mt-6 max-w-3xl leading-7 text-slate-700">
  {event.description}
</p>

<div className="mt-8">
  <SocietyEventInterestButton
    societyEventId={event.id}
    userId={user?.id ?? null}
    initialInterested={interested}
  />
</div>
</section>
</div>
```

</> TypeScript



Brief test steps

1. Run the SQL in Supabase SQL Editor.
2. In the `society_events` table, add or edit:
 - name
 - image_url
 - start_at
 - end_at
 - location
 - tag
 - short_description
 - description
3. Save the two updated page files.
4. Run:
npm run dev



5. Open `/societies`
 - confirm each society event card shows the image
 - confirm it shows `short_description`
 - confirm View details works
6. Open a society event detail page
 - confirm it shows image
 - confirm it shows `short_description` near the title
 - confirm it shows the full description lower down

Likely accessibility issues and fixes

1. Keep `short_description` short enough for easy scanning on cards.
2. Do not use description text as alt text for images. Keep alt as the event name.
3. Make the View details link more descriptive later if wanted:
View details for {event.name}
4. If `short_description` is missing in Supabase, the list page will look unappealing.

If you want, I can next give you the **exact Supabase table layout** for `society_events` in a clean copy-paste format so you can match every column properly.



when you receive a notification for a new event in your society, there is an open button make that button open the full event rather than just go to societies

I found the cause: the notification `href` is being written in SQL when a new society event is inserted, and right now it's hard-coded to `/societies`. I'm switching that stored link to the specific society event detail route so the existing Open button goes straight to the full event page.

Thought for 27s >

You only need to change the notification link that gets stored when a new `society_events` row is inserted. Supabase supports row-level Postgres triggers for this, and your society event detail page already uses a dynamic App Router route at `/societies/events/[id]`, so the notification `href` should point there instead of `/societies`. [Supabase +2](#)

Files to change

supabase/schema.sql



Code only

Replace the notification trigger function in `supabase/schema.sql`

```
create or replace function public.handle_new_society_event_notifications()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
begin
  insert into public.notifications (user_id, title, body, href)
  select
    sm.user_id,
    'New society event',
    new.name || ' was added for a society you joined.',
    '/societies/events/' || new.id::text
  from public.society_memberships sm
  where sm.society_id = new.society_id;

  return new;
end;
```

</> SQL



If you want existing old notifications fixed too, run this once in Supabase SQL Editor

```
update public.notifications n
set href = '/societies/events/' || se.id::text
from public.society_events se
where n.href = '/societies'
```

</> SQL



Brief test steps

1. Open Supabase SQL Editor.
2. Run the create or replace function SQL.
3. If needed, run the update SQL for old notifications too.



4. Make sure your test user is already joined to a society.
5. Insert a new row into `public.society_events` for that society.
6. Log into the app with that user.
7. Open My Account > Notifications.
8. Click Open on the new notification.
9. Confirm it now opens `/societies/events/[id]` and shows the full society e

SELF-CHECK: likely accessibility issues and fixes

1. The Open link text is a bit vague.
Better fix:
change "Open" to "Open event" or "View society event".
2. If multiple notifications exist, repeated "Open" links are less clear fo
Better fix:
use text like "View ETF Basics Night".
3. If a linked event is deleted later, the href may lead to a not found pag
Better fix:
keep the 404 page clear and add a message like "This society event is no

Your Next.js route can stay exactly as it is, because dynamic segments like `[id]` are the correct way to open a detail page from stored IDs. [Next.js +1](#)

       Sources

can you give me the new starter supabase schema with all the new updates made

Thought for 12s >

Replace your `supabase/schema.sql` with this full updated version:

```
create extension if not exists pgcrypto;

drop trigger if exists on_auth_user_created on auth.users;
drop trigger if exists on_society_event_created on public.society_events;

drop function if exists public.handle_new_user cascade;
drop function if exists public.handle_new_society_event_notifications cascade;
```

```
drop view if exists public.event_interest_admin_view cascade;
drop view if exists public.society_event_interest_admin_view cascade;
drop view if exists public.society_members_admin_view cascade;

drop table if exists public.helpdesk_messages cascade;
drop table if exists public.lost_found_items cascade;
drop table if exists public.notifications cascade;
drop table if exists public.society_event_interests cascade;
drop table if exists public.society_events cascade;
drop table if exists public.society_memberships cascade;
drop table if exists public.societies cascade;
drop table if exists public.event_interests cascade;
drop table if exists public.events cascade;
drop table if exists public.accessibility_preferences cascade;
drop table if exists public.map_locations cascade;
drop table if exists public.timetable_entries cascade;
drop table if exists public.profiles cascade;
drop table if exists public.timetable_groups cascade;

drop type if exists public.text_scale_option cascade;
drop type if exists public.lost_found_type cascade;

create type public.text_scale_option as enum ('sm', 'base', 'lg');
create type public.lost_found_type as enum ('lost', 'found');

create table public.timetable_groups (
  id uuid primary key default gen_random_uuid(),
  code text not null unique,
  title text not null,
  created_at timestamptz not null default now()
);

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null,
  email text not null unique,
  selected_timetable_group_id uuid references public.timetable_groups(id) on delete cascade,
  created_at timestamptz not null default now()
);

create table public.accessibility_preferences (
  user_id uuid primary key references auth.users(id) on delete cascade,
  high_contrast boolean not null default false,
  text_scale public.text_scale_option not null default 'base',
  updated_at timestamptz not null default now()
);

create table public.timetable_entries (
  id uuid primary key default gen_random_uuid(),
  group_id uuid not null references public.timetable_groups(id) on delete cascade,
  day_of_week integer not null check (day_of_week between 1 and 5),
  start_time time not null,
```



```
end_time time not null,  
module_code text not null,  
module_name text not null,  
lecturer_name text not null,  
room_name text not null,  
created_at timestamptz not null default now()  
);  
  
create table public.events (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  image_url text not null,  
  start_at timestamptz not null,  
  end_at timestamptz not null,  
  location text not null,  
  tag text not null,  
  short_description text not null,  
  description text not null,  
  created_at timestamptz not null default now()  
);  
  
create table public.event_interests (  
  id uuid primary key default gen_random_uuid(),  
  event_id uuid not null references public.events(id) on delete cascade,  
  user_id uuid not null references auth.users(id) on delete cascade,  
  created_at timestamptz not null default now(),  
  unique (event_id, user_id)  
);  
  
create table public.societies (  
  id uuid primary key default gen_random_uuid(),  
  name text not null unique,  
  tag text not null,  
  description text not null,  
  hero_image_url text not null,  
  created_at timestamptz not null default now()  
);  
  
create table public.society_memberships (  
  id uuid primary key default gen_random_uuid(),  
  society_id uuid not null references public.societies(id) on delete cascade,  
  user_id uuid not null references auth.users(id) on delete cascade,  
  created_at timestamptz not null default now(),  
  unique (society_id, user_id)  
);  
  
create table public.society_events (  
  id uuid primary key default gen_random_uuid(),  
  society_id uuid not null references public.societies(id) on delete cascade,  
  name text not null,  
  image_url text not null,  
  start_at timestamptz not null,
```

```
end_at timestampz not null,  
location text not null,  
tag text not null,  
short_description text not null,  
description text not null,  
created_at timestampz not null default now()  
);  
  
create table public.society_event_interests (  
  id uuid primary key default gen_random_uuid(),  
  society_event_id uuid not null references public.society_events(id) on delete c  
  user_id uuid not null references auth.users(id) on delete cascade,  
  created_at timestampz not null default now(),  
  unique (society_event_id, user_id)  
);  
  
create table public.notifications (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references auth.users(id) on delete cascade,  
  title text not null,  
  body text not null,  
  href text,  
  is_read boolean not null default false,  
  created_at timestampz not null default now()  
);  
  
create table public.map_locations (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  tag text not null,  
  latitude numeric(9, 6) not null,  
  longitude numeric(9, 6) not null,  
  description text,  
  created_at timestampz not null default now()  
);  
  
create table public.lost_found_items (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references auth.users(id) on delete cascade,  
  item_type public.lost_found_type not null,  
  title text not null,  
  description text not null,  
  location text not null,  
  contact_email text not null,  
  created_at timestampz not null default now()  
);  
  
create table public.helpdesk_messages (  
  id uuid primary key default gen_random_uuid(),  
  user_id uuid not null references auth.users(id) on delete cascade,  
  subject text not null,  
  category text not null,
```

```
message text not null,  
created_at timestamptz not null default now()  
);  
  
create or replace function public.handle_new_user()  
returns trigger  
language plpgsql  
security definer  
set search_path = public  
as $$  
begin  
    insert into public.profiles (id, full_name, email)  
    values (  
        new.id,  
        coalesce(new.raw_user_meta_data ->> 'full_name', 'New Student'),  
        new.email  
    );  
  
    insert into public.accessibility_preferences (user_id)  
    values (new.id);  
  
    return new;  
end;  
$$;  
  
create trigger on_auth_user_created  
after insert on auth.users  
for each row execute procedure public.handle_new_user();  
  
create or replace function public.handle_new_society_event_notifications()  
returns trigger  
language plpgsql  
security definer  
set search_path = public  
as $$  
begin  
    insert into public.notifications (user_id, title, body, href)  
    select  
        sm.user_id,  
        'New society event',  
        new.name || ' was added for a society you joined.',  
        '/societies/events/' || new.id::text  
    from public.society_memberships sm  
    where sm.society_id = new.society_id;  
  
    return new;  
end;  
$$;  
  
create trigger on_society_event_created  
after insert on public.society_events  
for each row execute procedure public.handle_new_society_event_notifications();
```

```
alter table public.profiles enable row level security;
alter table public.accessibility_preferences enable row level security;
alter table public.timetable_groups enable row level security;
alter table public.timetable_entries enable row level security;
alter table public.events enable row level security;
alter table public.event_interests enable row level security;
alter table public.societies enable row level security;
alter table public.society_memberships enable row level security;
alter table public.society_events enable row level security;
alter table public.society_event_interests enable row level security;
alter table public.notifications enable row level security;
alter table public.map_locations enable row level security;
alter table public.lost_found_items enable row level security;
alter table public.helpdesk_messages enable row level security;

create policy "profiles_select_own"
on public.profiles
for select
to authenticated
using (auth.uid() is not null and auth.uid() = id);

create policy "profiles_update_own"
on public.profiles
for update
to authenticated
using (auth.uid() is not null and auth.uid() = id)
with check (auth.uid() is not null and auth.uid() = id);

create policy "accessibility_select_own"
on public.accessibility_preferences
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "accessibility_insert_own"
on public.accessibility_preferences
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "accessibility_update_own"
on public.accessibility_preferences
for update
to authenticated
using (auth.uid() is not null and auth.uid() = user_id)
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "timetable_groups_public_read"
on public.timetable_groups
for select
to anon, authenticated
```

```
using (true);

create policy "timetable_entries_public_read"
on public.timetable_entries
for select
to anon, authenticated
using (true);

create policy "events_public_read"
on public.events
for select
to anon, authenticated
using (true);

create policy "event_interests_select_own"
on public.event_interests
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "event_interests_insert_own"
on public.event_interests
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "event_interests_delete_own"
on public.event_interests
for delete
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "societies_public_read"
on public.societies
for select
to anon, authenticated
using (true);

create policy "society_events_public_read"
on public.society_events
for select
to anon, authenticated
using (true);

create policy "society_memberships_select_own"
on public.society_memberships
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "society_memberships_insert_own"
on public.society_memberships
```

```
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "society_memberships_delete_own"
on public.society_memberships
for delete
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "society_event_interests_select_own"
on public.society_event_interests
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "society_event_interests_insert_own"
on public.society_event_interests
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "society_event_interests_delete_own"
on public.society_event_interests
for delete
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "notifications_select_own"
on public.notifications
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "notifications_update_own"
on public.notifications
for update
to authenticated
using (auth.uid() is not null and auth.uid() = user_id)
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "map_locations_public_read"
on public.map_locations
for select
to anon, authenticated
using (true);

create policy "lost_found_insert_own"
on public.lost_found_items
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);
```

```
create policy "lost_found_select_own"
on public.lost_found_items
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create policy "helpdesk_insert_own"
on public.helpdesk_messages
for insert
to authenticated
with check (auth.uid() is not null and auth.uid() = user_id);

create policy "helpdesk_select_own"
on public.helpdesk_messages
for select
to authenticated
using (auth.uid() is not null and auth.uid() = user_id);

create view public.event_interest_admin_view as
select
    e.name as event_name,
    p.full_name,
    p.email,
    ei.created_at as interested_at
from public.event_interests ei
join public.events e on e.id = ei.event_id
join public.profiles p on p.id = ei.user_id;

create view public.society_event_interest_admin_view as
select
    se.name as society_event_name,
    s.name as society_name,
    p.full_name,
    p.email,
    sei.created_at as interested_at
from public.society_event_interests sei
join public.society_events se on se.id = sei.society_event_id
join public.societies s on s.id = se.society_id
join public.profiles p on p.id = sei.user_id;

create view public.society_members_admin_view as
select
    s.name as society_name,
    p.full_name,
    p.email,
    sm.created_at as joined_at
from public.society_memberships sm
join public.societies s on s.id = sm.society_id
join public.profiles p on p.id = sm.user_id;

insert into public.timetable_groups (code, title)
```

```

values
    ('G1', 'Group 1'),
    ('G2', 'Group 2'),
    ('G3', 'Group 3');

insert into public.timetable_entries (
    group_id,
    day_of_week,
    start_time,
    end_time,
    module_code,
    module_name,
    lecturer_name,
    room_name
)
values
    ((select id from public.timetable_groups where code = 'G1'), 1, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G1'), 2, '12:00', '14:00',
    ((select id from public.timetable_groups where code = 'G1'), 4, '10:00', '12:00',
    ((select id from public.timetable_groups where code = 'G1'), 5, '14:00', '16:00',

    ((select id from public.timetable_groups where code = 'G2'), 1, '10:00', '12:00',
    ((select id from public.timetable_groups where code = 'G2'), 3, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G2'), 4, '13:00', '15:00',
    ((select id from public.timetable_groups where code = 'G2'), 5, '11:00', '13:00',

    ((select id from public.timetable_groups where code = 'G3'), 2, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G3'), 3, '12:00', '14:00',
    ((select id from public.timetable_groups where code = 'G3'), 4, '09:00', '11:00',
    ((select id from public.timetable_groups where code = 'G3'), 5, '13:00', '15:00',

insert into public.events (
    name,
    image_url,
    start_at,
    end_at,
    location,
    tag,
    short_description,
    description
)
values
    (
        'Welcome Fair',
        'https://images.unsplash.com/photo-1529156069898-49953e39b3ac?q=80&w=1200&auto=compress&format=webp',
        now() + interval '7 days',
        now() + interval '7 days 3 hours',
        'Main Hall',
        'Campus Life',
        'Meet societies, student services, and other first years.',
        'A fictional welcome fair for first year students. Explore student services,
    ),

```



```

(
  'Study Skills Bootcamp',
  'https://images.unsplash.com/photo-1522202176988-66273c2fd55f?q=80&w=1200&auto=compress&format=webp',
  now() + interval '10 days',
  now() + interval '10 days 2 hours',
  'Learning Centre',
  'Academic',
  'Time management, note taking, and exam prep tips.',
  'A practical workshop covering revision structure, note making, and weekly planning.',
),
(
  'Career Coffee Morning',
  'https://images.unsplash.com/photo-1515169067868-5387ec356754?q=80&w=1200&auto=compress&format=webp',
  now() + interval '14 days',
  now() + interval '14 days 2 hours',
  'Student Hub Café',
  'Careers',
  'Informal networking and CV advice.',
  'Meet the fictional careers team for a casual networking event with coffee and a careers advisor.',
);

insert into public.societies (
  name,
  tag,
  description,
  hero_image_url
)
values
(
  'Finance Society',
  'Careers',
  'Networking, trading games, and guest speakers for students interested in finance.',
  'https://images.unsplash.com/photo-1554224155-8d04cb21cd6c?q=80&w=1200&auto=compress&format=webp',
),
(
  'Debate Society',
  'Community',
  'Weekly debates, public speaking practice, and competitions.',
  'https://images.unsplash.com/photo-1515165562835-c3b8c2b9d4c0?q=80&w=1200&auto=compress&format=webp',
),
(
  'Tech Makers',
  'Innovation',
  'Friendly coding sessions and beginner workshops for practical projects.',
  'https://images.unsplash.com/photo-1516321318423-f06f85e504b3?q=80&w=1200&auto=compress&format=webp',
);

insert into public.society_events (
  society_id,
  name,
  image_url,
  start_at,
  end_at,
  description
)
values
(
  1,
  'Study Skills Bootcamp',
  'https://images.unsplash.com/photo-1522202176988-66273c2fd55f?q=80&w=1200&auto=compress&format=webp',
  '2026-05-01 18:00:00',
  '2026-05-01 20:00:00',
  'A practical workshop covering revision structure, note making, and weekly planning.'
),
(
  1,
  'Career Coffee Morning',
  'https://images.unsplash.com/photo-1515169067868-5387ec356754?q=80&w=1200&auto=compress&format=webp',
  '2026-05-08 10:00:00',
  '2026-05-08 12:00:00',
  'Meet the fictional careers team for a casual networking event with coffee and a careers advisor.'
),
(
  2,
  'Finance Society',
  'https://images.unsplash.com/photo-1554224155-8d04cb21cd6c?q=80&w=1200&auto=compress&format=webp',
  '2026-05-15 18:00:00',
  '2026-05-15 20:00:00',
  'Networking, trading games, and guest speakers for students interested in finance.'
),
(
  2,
  'Debate Society',
  'https://images.unsplash.com/photo-1515165562835-c3b8c2b9d4c0?q=80&w=1200&auto=compress&format=webp',
  '2026-05-22 18:00:00',
  '2026-05-22 20:00:00',
  'Weekly debates, public speaking practice, and competitions.'
),
(
  2,
  'Tech Makers',
  'https://images.unsplash.com/photo-1516321318423-f06f85e504b3?q=80&w=1200&auto=compress&format=webp',
  '2026-05-29 18:00:00',
  '2026-05-29 20:00:00',
  'Friendly coding sessions and beginner workshops for practical projects.'
);

```

```

end_at,
location,
tag,
short_description,
description
)
values
(
  (select id from public.societies where name = 'Finance Society'),
  'ETF Basics Night',
  'https://images.unsplash.com/photo-1559526324-4b87b5e36e44?q=80&w=1200&auto=f
now() + interval '5 days',
now() + interval '5 days 2 hours',
  'Room F12',
  'Workshop',
  'A beginner friendly introduction to ETFs, investing basics, and long term fi
  'An introductory session on ETFs, long term investing, and financial literacy
),
(
  (select id from public.societies where name = 'Debate Society'),
  'Mini Debate Tournament',
  'https://images.unsplash.com/photo-1517048676732-d65bc937f952?q=80&w=1200&aut
now() + interval '8 days',
now() + interval '8 days 3 hours',
  'Lecture Theatre 2',
  'Competition',
  'Friendly team debates with quick rounds designed to build confidence and pub
  'Students form quick teams and take part in friendly debate rounds with start
),
(
  (select id from public.societies where name = 'Tech Makers'),
  'Campus App Jam',
  'https://images.unsplash.com/photo-1515879218367-8466d910aaa4?q=80&w=1200&aut
now() + interval '12 days',
now() + interval '12 days 2 hours'
);

```

≡ ChatGPT ▾

🔗 ↗️ ...

```

  'A collaborative build session where students create useful digital ideas for
  'A beginner friendly build session where students prototype useful campus too
);

```

```

insert into public.map_locations (
  name,
  tag,
  latitude,
  longitude,
  description

```

SQL



```

( 'Library', 'Study', 55.349805, -6.260510, 'Quiet study floors, group rooms, an
('Student Hub', 'Support', 53.349220, -6.259750, 'Advice desk, careers support,

```

```
('Main Hall', 'Events', 53.350020, -6.261100, 'Large space for fairs, talks, an  
( 'Innovation Lab', 'Tech', 53.349420, -6.258900, 'Creative project and workshop
```



i need fictional student data that i can add to my supabase, and use for machine learning

I'm generating a synthetic dataset that matches your campus app theme, with useful columns for machine learning and no real personal data. I'm making it import-friendly for Supabase and adding target labels so you can train classification models straight away.

I've generated a clean synthetic dataset and I'm just balancing the target label a bit so it's more useful for training models, instead of making almost everyone "at risk."

Thought for 2m 30s >

I made you a fully fictional student dataset for Supabase and ML.

Files:

[fictional_student_ml_dataset.csv](#)

[student_ml_dataset_table.sql](#)

[student_ml_dataset_readme.txt](#)

What's inside:

- 500 synthetic students
- names and emails are fictional
- campus-style features like timetable group, attendance, study hours, event interest, society activity, helpdesk usage, accessibility settings
- ML targets included:
 - `target_at_risk` for binary classification
 - `target_performance_band` for multiclass classification
 - `estimated_final_mark` for regression

Best way to add it to Supabase:

1. Run [student_ml_dataset_table.sql](#) in the SQL Editor.
2. Open the `student_ml_dataset` table in Supabase.
3. Import [fictional_student_ml_dataset.csv](#).

A few useful columns for machine learning:

- attendance_rate
- study_hours_per_week
- avg_assignment_score
- avg_quiz_score
- engagement_score
- wellbeing_score
- commute_minutes
- part_time_job_hours
- late_submissions_count

If you want, I can also generate:

- a **larger version** with 5,000 rows
- a **cleaner version** just for classification
- a **messier realistic version** with missing values for data cleaning practice

